

Switch Transformers: 通过简单高效的稀疏性扩展至万亿参数模型

威廉·费德斯*

liamfedus@google.com

巴雷特佐夫*

barretzovh@google.com

诺姆·沙泽尔

noam@google.com

谷歌公司, 美国加利福尼亚州山景城, 邮编 94043

编辑: 亚历山大·克拉克

摘要

在深度学习中, 模型通常对所有输入复用相同的参数。混合专家 (MoE) 模型则打破了这一常规, 而是为每个输入样本选择不同的参数。其结果是形成一个稀疏激活的模型 —— 拥有数量惊人的参数, 但计算成本保持恒定。然而, 尽管 MoE 取得了一些显著的成功, 但其广泛应用却受到复杂性、通信成本和训练不稳定性的阻碍。我们通过引入 Switch Transformer 来解决这些问题。我们简化了 MoE 的路由算法, 并设计了具有直观改进的模型, 降低了通信和计算成本。我们提出的训练技术缓解了不稳定性, 并且首次证明可以用更低精度 (bfloat16) 格式训练大型稀疏模型。我们基于 T5-Base 和 T5-Large (Raffel 等人, 2019) 设计模型, 在相同计算资源下, 预训练速度提升高达 7 倍。这些改进延伸到多语言场景中, 在所有 101 种语言上, 我们都观测到其性能超过 mT5-Base 版本。最后, 我们通过在 “Colossal Clean” 上预训练高达万亿参数的模型, 推进了当前语言模型的规模。

“Colossal Clean Crawled Corpus” 超大规模清洁爬取语料库) 上预训练了高达万亿参数的模型, 并且相比 T5-XXL 模型实现了 4 倍的速度提升。

关键词: 专家混合、自然语言处理、稀疏性、大规模机器学习、分布式计算

。 同登吾辞

Switch Transformer 的 JAX 代码和所有模型检查点可在

Switch Transformer 的 TensorFlow 代码可在

<https://github.com/tensorflow/mesh/blob/>

©2022 威廉·费德斯、巴雷特·佐夫和诺姆·沙泽尔。

许可证: CC-BY 4.0, 详见<https://creativecommons.org/licenses/by/4.0/>。归因要求详见<http://jmlr.org/papers/v23/21-0998.html>。

ar
Xiv
:
21
01.
03
96
1v
3
[cs
.L
G]
20
22
年
6
月
16
日

目录

1 引言	3
2 切换 Transformer	4
2.1 简化稀疏路由	5
2.2 高效稀疏路由	6
2.3 整合所有部分：切换 Transformer	8
2.4 改进的训练和微调技术	8
3 缩放特性	11
3.1 基于步骤的缩放结果	12
3.2 基于时间的缩放结果	13
3.3 与更大的密集模型的缩放对比	13
4 下游结果	14
4.1 微调	14
4.2 蒸馏	16
4.3 多语言学习	17
5 设计具有数据、模型和专家并行性的模型	19
5.1 数据并行性	20
5.2 模型并行性	20
5.3 模型和数据并行性	21
5.4 专家和数据并行性	22
5.5 专家、模型和数据并行性	22
5.6 迈向万亿参数模型	22
6 相关工作	24
7 讨论	25
8 未来工作	26
9 结论	27
A 注意力的切换机制	27
B 用“不落下任何标记”防止标记丢失	29
C 鼓励跨专家的探索	29
D 低计算量情况下的切换 Transformer	29
E 上游与下游模型性能的关系	32
F 切换 Transformer 的伪代码	33

引言

大规模训练已成为打造灵活且强大的神经语言模型的有效途径 (Radford 等人, 2018; Kaplan 等人, 2020; Brown 等人, 2020)。简单的架构在充足的计算资源、数据集规模和参数数量的支持下, 超越了更复杂的算法 (Sutton, 2019)。Radford 等人 (2018)、Raffel 等人 (2019) 以及 Brown 等人 (2020) 所采用的方法是扩大密集激活的 Transformer (Vaswani 等人, 2017) 的模型规模。尽管这种方法有效, 但计算量也极大 (Strubell 等人, 2019)。受模型规模成功的启发, 同时为了追求更高的计算效率, 我们提出了一种稀疏激活的专家模型: Switch Transformer。在我们的模型中, 稀疏性源于对每个输入样本仅激活神经网络权重的一个子集。

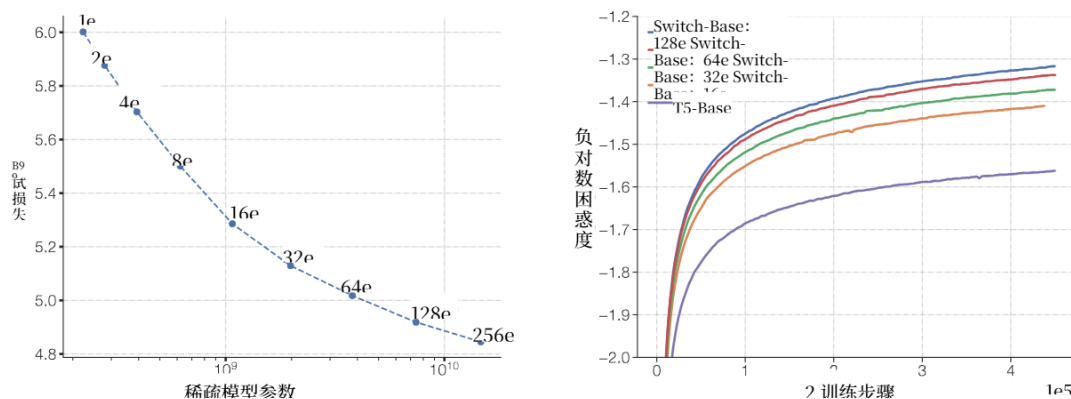


图 1: Switch Transformer 的扩展性和样本效率。左图: 稀疏性不断提高 (专家数量更多) 的 Switch Transformer 的扩展特性。右图: 在相同计算预算下, Switch Transformer 与 T5 模型 (Raffel 等人, 2019) 的负对数困惑度对比。

稀疏训练是一个活跃的研究和工程领域 (Gray 等人, 2017; Gale 等人, 2020), 但时至今日, 机器学习库和硬件加速器仍然适用于密集矩阵乘法。为了获得高效的稀疏算法, 我们从混合专家 (MoE) 范式 (Jacobs 等人, 1991; Jordan 和 Jacobs, 1994; Shazeer 等人, 2017) 入手, 并对其进行简化, 以实现训练稳定性和计算优势。MoE 模型在机器翻译方面取得了显著成功 (Shazeer 等人, 2017、2018; Lepikhin 等人, 2020), 然而, 其广泛应用受到复杂性、通信成本和训练不稳定性的阻碍。

我们解决了这些问题, 并且不止于翻译, 还发现这类算法在自然语言领域具有广泛的价值。我们在一系列不同的自然语言任务上, 以及在自然语言处理的三个领域 (预训练、微调与多任务训练) 中, 都测量到了出色的扩展性。虽然这项工作侧重于规模, 但我们 also 表明, Switch Transformer 架构不仅在超级计算机领域表现出色, 而且还能

即使只有几个计算核心，这也是有益的。此外，我们的大型稀疏模型可以被蒸馏（Hinton 等人，2015）成小型密集版本，同时保留稀疏模型 30% 的质量提升。我们的贡献如下：

- Switch Transformer 架构简化并改进了混合专家模型。
 - 缩放特性以及与经过深度调优的 T5 模型（Raffel 等人，2019 年）的基准测试 —— 我们在保持每个 token 的 FLOPS 相同的情况下，实现了 7 倍以上的预训练速度提升。我们进一步表明，即使在计算资源有限的情况下（仅使用少至两个专家），这种改进仍然成立。
 - 成功将稀疏预训练模型和专门微调模型提炼为小型密集模型。我们将模型大小缩减了高达 99%，同时保留了大型稀疏教师模型 30% 的质量提升。
- 改进的预训练和微调技术：（1）选择性精度训练，支持使用更低的 bfloat16 精度进行训练；（2）一种初始化方案，能够扩展到更多数量的专家；（3）增强的专家正则化，改善稀疏模型的微调和多任务训练。
 - 一项关于多语言数据预训练收益的测量显示，所有 101 种语言均获得了普遍提升，其中 91% 的语言相较于 mT5 基准（Xue 等人，2020 年）实现了 4 倍及以上的速度提升。
 - 通过高效结合数据并行、模型并行和专家并行，扩大神经语言模型的规模，从而创建出拥有多达一万亿参数的模型。这些模型将经过深度调优的 T5-XXL 基准模型的预训练速度提升了 4 倍。

切换 Transformer

Switch Transformer 的指导性设计原则是通过一种简单且计算高效的方式最大化 Transformer 模型（Vaswani 等人，2017）的参数数量。Kaplan 等人（2020）对规模带来的益处进行了详尽研究，发现模型大小、数据集大小和计算预算存在幂律缩放关系。重要的是，这项研究主张在相对少量的数据上训练大型模型，认为这是计算最优的方法。

基于这些结果，我们研究了第四个方向：在保持每个样本的浮点运算（FLOPs）不变的情况下增加参数数量。我们的假设是，参数数量与所执行的总计算量无关，是一个单独的重要扩展维度。我们通过设计一种稀疏激活模型来实现这一点，该模型能高效利用为密集矩阵乘法设计的硬件，如 GPU 和 TPU。我们的研究重点是 TPU 架构，但这类模型也可以在 GPU 集群上进行类似的训练。在我们的分布式训练设置中，稀疏激活层会将独特的权重分配到不同的设备上。因此，模型的权重会随着设备数量的增加而增加，同时每个设备上的内存和计算量都保持在可管理的范围内。

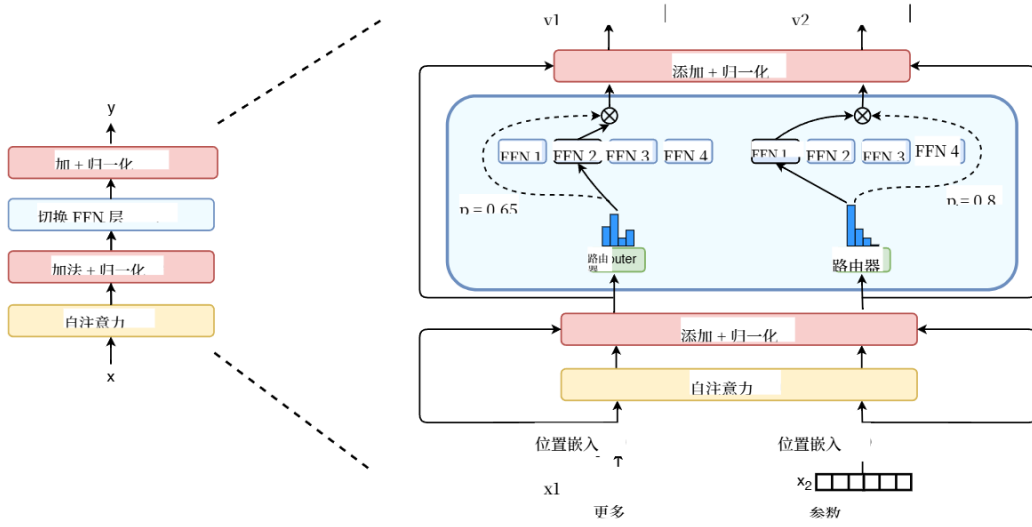


图 2: Switch Transformer 编码器块示意图。我们用稀疏的 Switch FFN 层（浅蓝色）替换了 Transformer 中存在的密集前馈网络（FFN）层。该层独立作用于序列中的标记。我们绘制了两个标记（下方的 $x_1 = \text{"More"}$ 和 $x_2 = \text{"参数"}$ ）通过四条线（实线）路由到四个 FFN 专家的过程，其中路由器独立路由每个标记。Switch FFN 层返回所选 FFN 的输出乘以路由器门控值（虚线）。

2.1 简化稀疏路由

专家混合路由。Shazeer 等人（2017）提出了一种自然语言专家混合（MoE）层，该层接收令牌表示 x 作为输入，然后将其路由到从 N 个专家集合 $E_i(x)_{i=1}^N$ 中选出的最佳确定的前 k 个专家。路由器变量 W_r 生成 logits $h(x) = W_r \cdot x$ ，这些 logits 通过该层可用的 N 个专家上的 softmax 分布进行归一化。专家 i 的门控值由下式给出：

$$p_i(x) = \frac{e^{h(x)_i}}{\sum_{j=1}^N e^{h(x)_j}}.$$

选择前 k 个门控值用于路由令牌 x 。如果 T 是所选前 k 个索引的集合，那么该层的输出计算就是每个专家对令牌的计算与门控值的线性加权组合，

$$y = \sum_{i \in T} p_i(x) E_i(x).$$

Switch Routing: 对混合专家模型的再思考。Shazeer 等人（2017）推测，为了使路由函数获得非平凡梯度，路由到 $k > 1$ 专家是必要的。作者们认为，如果没有能力比较至少两个专家，学习路由的方法就不会奏效。Ramachandran 和 Le（2018）进一步指出

研究了 top-k 决策，发现对于具有许多路由层的模型而言，较低层中较高的 k 值至关重要。与这些观点相反，我们采用了一种简化策略，即只路由到单个专家。我们证明这种简化保留了模型质量，减少了路由计算量，并且表现更优。这种 $k = 1$ 路由策略后来被称为切换层（Switch layer）。请注意，对于 MoE 和切换路由（Switch Routing），公式 2 中的门值 $\pi_i(x)$ 都保证了路由器的可微性。

Switch 层的好处有三个方便：（1）减少了路由器的计算量，因为我们只需要将一个令牌路由到单个专家。（2）批处理大小（专家容量）为

每个专家的批量大小（专家容量）至少可以减半，因为每个令牌只会被路由到单个专家。³

（3）路由实现得到简化，通信成本有所降低。图 3 展示了具有不同专家容量系数的路由示例。

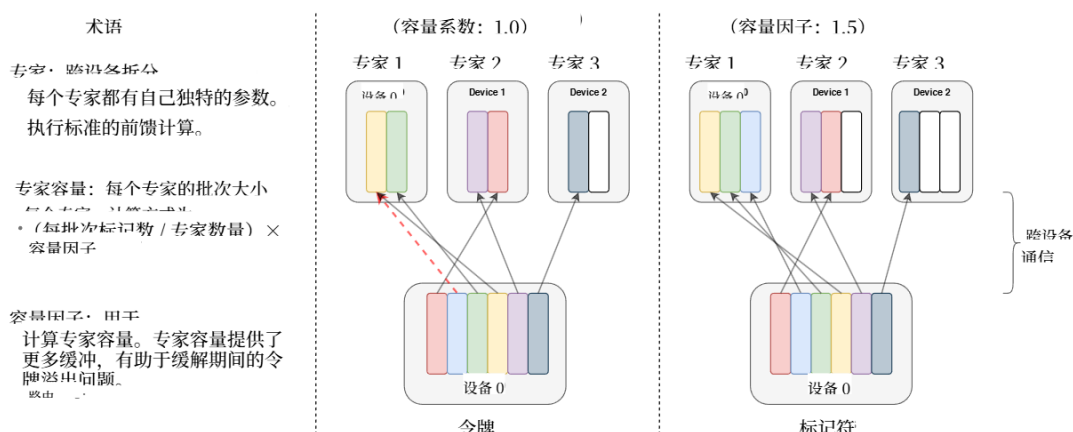


图 3：令牌路由动态示意图。每个专家处理固定批次大小的令牌，该批次大小由容量因子调节。每个令牌被路由到具有最高路由器概率的专家，但每个专家的固定批次大小为（总令牌数 / 专家数量） $\times$ 容量因子。如果令牌分配不均匀，某些专家将出现溢出（用红色虚线表示），导致这些令牌不会被该层处理。较大的容量因子可以缓解这种溢出问题，但也会增加计算和通信成本（用填充的白色 / 空槽表示）。

2.2 高效稀疏路由

我们使用 Mesh-Tensorflow (MTF) (Shazeer 等人, 2018)，这是一个库，其语义和 API 与 Tensorflow (Abadi 等人, 2016) 相似，有助于实现高效的分布式数据和模型并行架构。它通过将物理核心集抽象为逻辑处理器网络来实现这一点。然后，张量和计算可以按命名维度进行分片，从而便于跨维度对模型进行轻松分区。我们在设计模型时考虑了 TPU，TPU 需要静态声明大小。下面我们将描述我们的分布式 Switch Transformer 实现。

³ 技术描述参见第 2.2 节。

分布式交换机实现。我们所有的张量形状都是在编译时静态确定的，但由于训练和推理时的路由决策，我们的计算是动态的。因此，一个重要的技术考量是如何设置专家容量。专家容量——即每个专家处理的令牌数量——是通过将批次中的令牌数量平均分配到专家数量上，然后再乘以一个容量因子来设置的，

$$= \left(\frac{\text{tokens per batch}}{\text{number of experts}} \right) \times \text{capacity factor}$$

容量因子大于 1.0 会产生额外的缓冲，以应对令牌在专家之间未能完美平衡的情况。如果过多令牌被路由到某个专家（后文称为丢弃令牌），则会跳过计算，令牌表示会通过残差连接直接传递到下一层。然而，增加专家容量并非没有缺点，因为过高的值会导致计算和内存的浪费。这种权衡在图 3 中有解释。从经验上看，我们发现确保较低的丢弃令牌率对于稀疏专家模型的扩展很重要。在所有实验中，我们没有发现丢弃令牌的数量与专家数量之间存在任何相关性（(typically < 1%) 1%）。使用具有足够高系数的辅助负载平衡损失（下一节）可确保良好的负载平衡。我们在表 1 中研究了这些设计决策对模型质量和速度的影响。

一种可微分的负载均衡损失。为了促进专家之间的负载均衡，我们添加了一个辅助损失（Shazeer 等人，2017 年，2018 年；Lepikhin 等人，2020 年）。正如 Shazeer 等人（2018 年）、Lepikhin 等人（2020 年）所述，Switch Transformers 简化了 Shazeer 等人（2017 年）的原始设计，后者具有单独的负载均衡损失和重要性加权损失。对于每个 Switch 层，该辅助损失在训练期间被添加到模型总损失中。给定 N 个专家（索引从 $i = 1$ 到 N ）和一个包含 T 个标记的批次 B ，辅助损失计算为向量 f 和 P 之间的缩放点积，

$$\text{loss} = \alpha \cdot N \cdot \sum_{i=1}^N f_i \cdot P_i \quad (4)$$

其中 f_i 是分配给专家 i 的标记比例，

$$f_i = \frac{1}{T} \sum_{x \in B} \mathbf{1}\{\text{argmax}(x) = i\}$$

而 P_i 是为专家 i 分配的路由器概率比例²

$$P_i = \frac{1}{T} \sum_{x \in B} p_i(x). \quad (6)$$

由于我们寻求在 N 个专家之间对这一批标记进行均匀路由，我们希望两个向量都具有 $1/N$ 的值。式 4 的辅助损失鼓励均匀路由，因为它在均匀分布下被最小化。该目标也可以被微分，如下

一个潜在的混淆点： $p_i(x)$ 是将令牌 x 路由到专家 i 的概率， P_i 是在批次 B 的所有令牌中分配给专家 i 的概率占比。

P 向量是可微的，但 f 向量不可微。最终损失乘以专家数量 N，以在专家数量变化时保持损失恒定，这是因为在均匀路由下 $\nabla_{\mathbf{P}_i} \sum_{i=1}^N (f_i \cdot P_i) = \nabla_{\mathbf{P}_i} \sum_{i=1}^N (\frac{1}{N} \cdot \frac{1}{N}) = \frac{1}{N}$ 。最后，超参数 α 是这些辅助损失的乘法系数；在整个研究中，我们使用了 $\alpha = 10^{-2}$ ，它足够大以确保负载均衡，同时又足够小，不会淹没主要的交叉熵目标。我们以 10 的幂次在从 10^{-1} 到 10^{-5} 的范围内扫描超参数 α ，发现 10^{-2} 能快速平衡负载，且不会干扰训练损失。

2.3 整合所有部分：Switch Transformer

我们对 Switch Transformer 的首次测试始于在 “Colossal Clean Crawled Corpus” (C4) 上进行预训练，该语料库在 (Raffel 等人, 2019) 中被提出。对于我们的预训练目标，我们使用掩码语言建模任务 (Taylor, 1953; Fedus 等人, 2018; Devlin 等人, 2018)，在该任务中，模型被训练来预测缺失的标记。在我们的预训练设置中，按照 Raffel 等人 (2019) 确定的最优方式，我们丢弃 15% 的标记，然后用一个单独的标记符替换被掩码的序列。为了比较我们的模型，我们记录了负对数困惑度。4 在本文的所有表格中， $\uparrow$ 表示该指标的值越高越好， $\downarrow$ 则相反。本研究中所有被研究模型的比较情况见表 9。

表 1 展示了 Switch Transformer 与 MoE Transformer 的直接对比。我们的 Switch Transformer 模型在 FLOP 上与 “T5-Base” (Raffel 等人, 2019 年) 相匹配 (对每个令牌应用相同的计算量)。使用 top-2 路由的 MoE Transformer 有两个专家，每个专家都对每个令牌应用单独的 FFN，因此其 FLOPS 更大。所有模型都在相同的硬件上以相同的步数进行训练。请注意，MoE 模型的容量因子从 2.0 降至 1.25 实际上会减慢

在上述实验设置中下降 (从 840 到 790)，这是出乎意料的。5

我们从表 1 中强调三个关键发现：(1) 在速度 - 质量基础上，Switch Transformer 的性能优于精心调整的密集模型和 MoE Transformer。在固定的计算量和实际时间下，Switch Transformer 取得了最佳结果。(2) Switch Transformer 的计算占用小于对应的 MoE 模型。如果我们增大其规模以匹配 MoE Transformer 的训练速度，会发现它在每一步的性能上也优于所有 MoE 模型和密集模型。(3) Switch Transformer 在较低的容量因子 (1.0、1.25) 下表现更好。较小的专家容量表明，在大型模型机制中，当模型内存非常紧张时，容量因子会尽可能小。

2.4 改进的训练和微调技术

稀疏专家模型可能会比普通 Transformer 带来更多训练难题。这是因为这些层中的每一层都存在硬切换 (路由) 决策，从而可能导致不稳定性。此外，像 bfloat16 (Wang 和 Kanwar, 2019) 这样的低精度格式会加剧这些问题。

我们在这个指标中使用以 e 为底的对数，因此单位是奈特

请注意，速度测量结果既取决于算法，也与实现细节有关。Switch Transformer 相较于 MoE 减少了必要的计算量 (这是算法层面的改进)，但最终的速度差异还会受到底层优化的影响 (这是实现层面的因素)。

Model	Capacity Factor	Quality after 100k steps (↑) (Neg. Log Perp.)	Time to Quality Threshold (↓) (hours)	Speed (↑) (examples/sec)
T5-Base	—	-1.731	Not achieved [†]	1600
T5-Large	—	-1.550	131.1	470
MoE-Base	2.0	-1.547	68.7	840
Switch-Base	2.0	-1.554	72.8	860
MoE-Base	1.25	-1.559	80.7	790
Switch-Base	1.25	-1.553	65.0	910
MoE-Base	1.0	-1.572	80.1	860
Switch-Base	1.0	-1.561	62.8	1000
Switch-Base+	1.0	-1.534	67.6	780

表 1: Switch 与 MoE 的基准测试。对比 Switch Transformer 相较于 MoE Transformer 和 T5 密集型基准模型在每步和每时间单位上的优势。我们通过负对数困惑度以及达到任意选定的质量阈值（负对数 $P_{\text{Perp.}} = -1.50$ ）所需的时间来衡量质量。所有 MoE 和 Switch Transformer 模型均使用 128 个专家，且每隔一个前馈层设置专家。对于 Switch-Base+，我们通过将模型隐藏层大小从 768 增加到 896、头数从 14 增加到 16，来增大模型规模，直至其速度与 MoE 模型相当。所有模型均在相同计算量（32 个核心）和相同硬件（TPUv3）上进行训练。此外需要注意的是，所有模型都需要超过 10 万步的预训练才能达到 -1.50 的水平阈值。[†] T5-Base 在模型训练的 10 万步内未达到这一负对数困惑度。

在我们路由的 softmax 计算中。我们在此描述了训练难点以及为实现稳定且可扩展的训练而采用的克服这些难点的方法。

大型稀疏模型的选择性精度。模型不稳定性阻碍了使用高效的 bfloat16 精度进行训练的能力，因此，Lepikhin 等人（2020）在其 MoE Transformer 中全程使用 float32 精度进行训练。然而，我们发现，通过在模型的局部部分选择性地转换为 float32 精度，可以在不产生 float32 张量昂贵通信成本的情况下实现稳定性。这种技术与现代混合精度训练策略一致，在现代混合精度训练策略中，模型的某些部分和梯度更新以更高的精度进行（Micikevicius 等人，2017）。表 2 显示，我们的方法在赋予 float32 训练稳定性的同时，速度几乎与 bfloat16 训练相当。

为实现这一点，我们将路由器输入转换为 float32 精度。路由器函数以令牌作为输入，并生成用于专家计算的选择和重组的分配张量与组合张量（详情请参见附录中的代码块 15）。重要的是，float32 精度仅在路由器函数体内使用——用于该设备本地的计算。由于生成的分配张量和组合张量在函数末尾会被重新转换为 bfloat16 精度，因此不会产生庞大的 float32 张量。

Model (precision)	Quality (Neg. Log Perp.) (↑)	Speed (Examples/sec) (↑)
Switch-Base (float32)	-1.718	1160
Switch-Base (bfloat16)	-3.780 [<i>diverged</i>]	1390
Switch-Base (Selective precision)	-1.716	1390

表 2: 选择性精度。我们将局部路由操作转换为 float32, 同时在其他地方保留 bfloat16 精度, 以稳定模型, 同时达到与 (不稳定的) bfloat16 精度训练几乎相同的速度。我们在训练早期的固定步数后测量 32 专家模型的质量及其速度性能。对于 float32 精度的 Switch-Base 模型和采用选择性精度的模型, 我们发现它们具有相似的学习动态。

通过全对全通信操作进行广播, 但我们仍然受益于 float32 更高的稳定性。

为了稳定性而采用更小的参数初始化。适当的初始化对于深度学习中的成功训练至关重要, 我们尤其发现这一点对于 Switch Transformer 来说是成立的。我们通过对截断正态分布中抽取元素来初始化权重矩阵, 该分布的均值为 $\mu = 0$, 标准差为 $\sigma = \sqrt{s/n}$, 其中 s 是一个尺度超参数, n 是权重张量中的输入单元数量 (例如扇入)。⁶

作为针对不稳定性的额外补救措施, 我们建议将默认的 Transformer 初始化缩放系数 $s = 1.0$ 减小 10 倍。在我们的实验中, 这一做法既能提高模型质量, 又能降低训练不稳定的可能性。表 3 衡量了训练初期模型质量的提升和方差的减少情况。我们发现, 以负对数困惑度 (Neg. Log Perp.) 衡量的平均模型质量得到了显著提升, 且不同运行之间的方差也大幅降低。此外, 这种初始化方案对跨越多个数量级的模型都具有广泛的有效性。我们采用相同的方法, 能够稳定地训练从 2.23 亿参数的基线模型到超过 1 万亿参数的超大型模型等各类规模的模型。

表 3: 减小初始化规模可提高稳定性。减小初始化规模能带来更好的模型质量, 并使 Switch Transformer 的训练更稳定。在此, 我们记录了 32 个专家模型在 3.5k 步后 (各 3 个随机种子) 的模型质量平均值和标准差, 该质量通过负对数困惑度来衡量。

⁶与平均值相差超过两个标准差的值会被重新采样。

对大型稀疏模型进行正则化。我们的论文探讨了自然语言处理中一种常见方法，即先在大型语料库上进行预训练，然后在更小的下游任务（如文本摘要或问答）上进行微调。一个自然出现的问题是过拟合，因为许多微调任务的样本数量很少。在对标准 Transformer 模型进行微调时，Raffel 等人（2019）在每一层都使用了 dropout（Srivastava 等人，2014）来防止过拟合。我们的 Switch Transformer 模型比浮点运算量相当的密集型基线模型拥有多得多的参数，这可能会导致在这些更小的下游任务上出现更严重的过拟合。

Model (dropout)	GLUE	CNNDM	SQuAD	SuperGLUE
T5-Base (d=0.1)	82.9	19.6	83.5	72.4
Switch-Base (d=0.1)	84.7	19.1	83.7	73.0
Switch-Base (d=0.2)	84.4	19.2	83.9	73.2
Switch-Base (d=0.3)	83.9	19.6	83.4	70.7
Switch-Base (d=0.1, ed=0.4)	85.2	19.6	83.7	73.0

表 4: 微调正则化结果。在对 C4 数据集 340 亿 token 预训练的 Switch Transformer 模型进行微调时，对 dropout 率进行了一系列测试（数值越高越好）。我们发现，在所有非专家层使用较低的标准 dropout 率，同时在专家前馈层使用大得多的 dropout 率，效果最佳。

因此，我们提出了一种在微调期间缓解此问题的简单方法：增加专家内部的 dropout，我们将其命名为专家 dropout。在微调期间，我们仅在每个专家层的中间前馈计算中显著提高 dropout 率。表 4 展示了我们的专家 dropout 方案的结果。我们发现，简单地在所有层中提高 dropout 会导致性能下降。然而，在非专家层设置较小的 dropout 率（0.1），在专家层设置大得多的 dropout 率（0.4），则会在四个较小的下游任务上带来性能提升。

缩放特性

我们对 Switch Transformer 架构在预训练过程中的缩放特性进行了研究。根据 Kaplan 等人（2020 年）的研究，我们考虑的是一种模型既不受计算预算限制，也不受数据量限制的情况。为了避免数据瓶颈，我们使用了包含超过 1800 亿目标标记的大型 C4 语料库（Raffel 等人，2019 年），并训练到观察到收益递减为止。

专家数量是扩展我们模型的最有效维度。增加专家数量能使计算成本大致保持稳定，因为无论可供选择的专家数量有多少，模型每个标记只选择一位专家。不过，路由机制必须计算更多专家的概率分布，这是一种轻量级计算，成本为 $O(d_{model} \times num)$ 个专家，其中 d_{model} 是嵌入维度。

各层之间传递的标记。在本节中，我们在固定计算预算的情况下，从步骤基准和时间基准两方面考虑缩放特性。

3.1 基于步数的扩展结果

图 4 展示了在固定步数下训练所有模型时，专家数量带来的持续扩展收益。我们观察到一个明显的趋势：当保持每个 token 的 FLOPS 固定时，拥有更多参数（专家）会加快训练速度。左图展示了稀疏模型参数与测试损失之间一致的扩展特性（在每个 token 的 FLOPS 固定的情况下）。这揭示了沿着稀疏模型参数这一额外维度进行扩展的优势。右图衡量了一个密集模型变体和四个 FLOP 匹配的稀疏变体的样本效率。我们发现，增加专家数量会使模型的样本效率更高。我们的 Switch-Base 64 专家模型在 450k 步时达到了 T5-Base 模型在 60k 步时的相同性能，这在步长时间方面是 7.5 倍的加速。此外，与 Kaplan 等人（2020）的研究结果一致，我们发现更大的模型也具有更高的样本效率——在固定数量的观测 token 下学习速度更快。

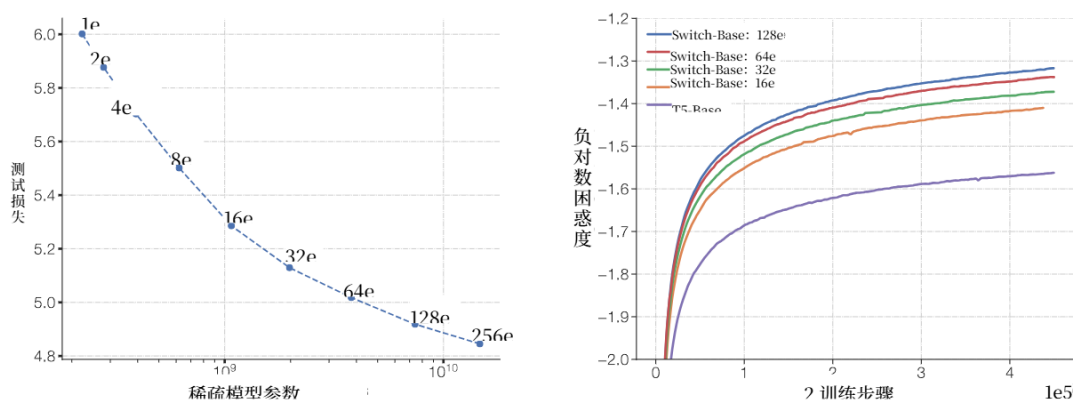


图 4: Switch Transformer 的缩放特性。左图：我们通过缩放专家数量来衡量随着参数增加，以困惑度为指标的质量提升。左上角的点对应具有 2.23 亿参数的 T5-Base 模型。从左上角到右下角，我们将专家数量从 2 个、4 个、8 个依次翻倍，直到右下角具有 147 亿参数的 256 个专家模型的点。尽管所有模型使用相同的计算预算，但我们观察到随着专家数量的增加，模型性能有持续的提升。右图：每一步的负对数困惑度随专家数量的变化。紫色线显示的是密集基线，我们注意到 Switch-Base 模型的样本效率有所提高。

3.2 基于时间的扩展结果

图 4 表明，在按步骤计算的情况下，随着专家数量的增加，性能会持续提升。虽然我们的模型每个令牌的 FLOPS 数量与基准模型大致相同，但 Switch Transformers 会在设备间产生额外的通信成本，以及路由机制的额外计算量。因此，在按步骤计算时观察到的样本效率提升，并不一定能转化为按实际时钟时间衡量的更好模型质量。这就引出了一个问题：

在固定的训练时长和计算预算下，应该训练稠密模型还是稀疏模型？

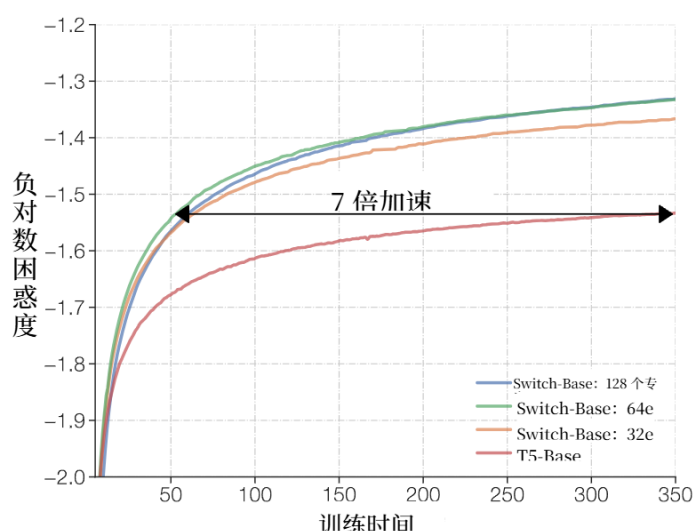


图 5: Switch Transformer 的速度优势。所有模型均在 32 个 TPUv3 核心上训练，每个样本的 FLOPs 相同。在计算量和训练时间固定的情况下，Switch Transformer 的性能显著优于密集型 Transformer 基准模型。我们的 64 专家 Switch-Base 模型仅用 T5-Base 七分之一的时间就达到了同等质量，并且性能还在持续提升。

图 5 和图 6 解决了这个问题。图 5 将预训练模型的质量作为时间的函数进行衡量。在固定的训练时长和计算预算下，Switch Transformer 带来了显著的速度提升。在这种情况下，我们的 Switch-Base 64 专家模型的训练时间仅为 T5-Base 达到相似困惑度所需时间的七分之一。

3.3 与更大的密集模型的缩放对比

上述分析表明，计算量相当的密集模型性能不及对应的 Switch 模型。图 6 考虑了另一种情况：如果我们将资源分配给更大的密集模型会怎样？我们现在就进行这样的对比，将 Switch-Base 与下一个强劲的基线模型 T5-Large 进行比较。但尽管 T5-Large 每 token 应用的计算量（FLOPs）是前者的 3.5 倍，

Switch-Base 仍然具有更高的样本效率，并实现了 2.5 倍的速度提升。此外，只需设计一个新的、更大的稀疏版本 Switch-Large（其 FLOP 与 T5-Large 相当），就能获得更多收益。我们将在接下来的部分中进行相关操作，并展示其更优的缩放性能和微调效果。

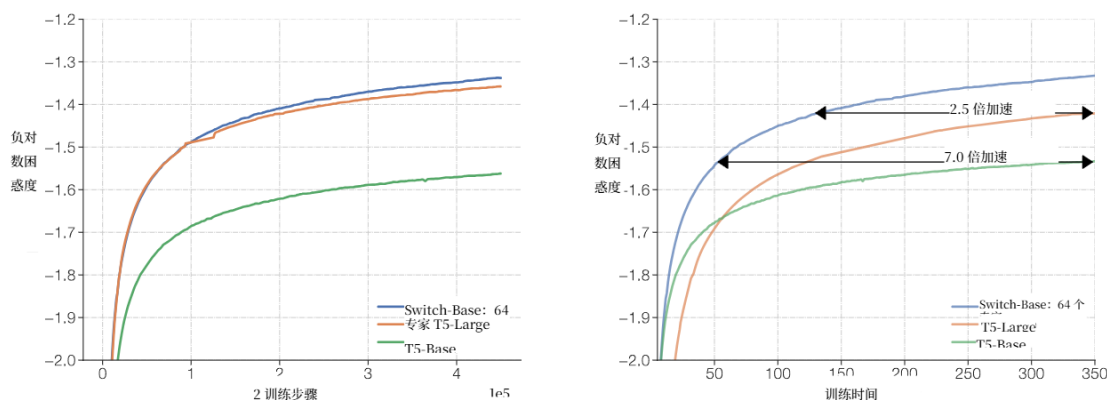


图 6：使用 Switch 层或标准密集模型缩放来扩展 Transformer 模型。左图：Switch-Base 比 T5-Base 和 T5-Large 变体的样本效率更高，后两者每个标记的 FLOPS（浮点运算次数）多 3.5 倍。右图：与之前一样，在实际时钟时间上，我们发现 Switch-Base 仍然更快，比 T5-Large 快 2.5 倍。

下游结果

第 3 节展示了预训练时卓越的缩放特性，但我们现在验证这些优势是否转化为下游任务中语言学习能力的提升。我们首先在多种自然语言处理任务上进行微调。接下来，我们研究通过将稀疏模型蒸馏到小型且易于部署的密集基线模型中，将其内存占用减少 90% 以上。最后，我们在本节中通过多任务、多语言场景衡量改进效果，结果表明 Switch Transformers 是强大的多任务学习者，在所有 101 种语言上均优于多语言 T5-base 模型。

4.1 微调

用于微调的基线模型和 Switch 模型。我们的基线模型是经过高度调优的 223M 参数 T5-Base 模型和 739M 参数 T5-Large 模型（Raffel 等人，2019）。对于这两个版本，我们设计了一个 FLOP 匹配的 Switch Transformer，它具有更多参数，详情总结在表 9 中。⁷ 我们的基线模型与 Raffel 等人（2019）的模型略有不同，因为我们在改进的 C4 语料库上进行预训练，该语料库去除了示例内的文本重复，从而提高了作为预训练任务的有效性（Lee 等人）。

⁷ FLOPS 的计算是按照 Kanlan 等人（2020 年）的方法针对前向传播进行的。

(2021 年)。在我们的协议中，我们使用每批次 220 (1,048,576) 个令牌进行预训练，共训练 550k 步，总计 5760 亿个令牌。然后，我们在多种任务上进行微调，所有层的 dropout 率为 0.1，但 Switch 层除外，其 dropout 率为 0.4 (见表 4)。我们使用 100 万的批次大小进行 16k 步的微调，对于每个任务，我们每 200 步评估一次模型质量，并报告在验证集上计算出的最佳性能。

微调任务和数据集。我们选择了用于探测语言能力的任务，包括问答、摘要以及关于世界的知识。语言基准 GLUE (Wang 等人, 2018) 和 SuperGLUE (Wang 等人, 2019) 被当作复合混合任务来处理，所有任务按照各自包含的标记数量比例进行融合。这些基准包含多种任务，如情感分析 (SST2)、词义消歧 (WIC)、句子相似度 (MRPC、STS-B、QQP)、自然语言推理 (MNLI、QNLI、RTE、CB)、问答 (MultiRC、RECORD、BoolQ)、共指消解 (WNLI、WSC)、句子补全 (COPA) 以及句子可接受性 (CoLA)。CNNDM (Hermann 等人, 2015) 和 BBC XSum (Narayan 等人, 2018) 数据集用于衡量文章摘要能力。问答能力通过 SQuAD 数据集 (Rajpurkar 等人, 2016) 和 ARC 推理挑战 (Clark 等人, 2018) 来探测。并且，如同 Roberts 等人 (2020) 的研究，我们通过在三个闭卷问答数据集上进行微调来评估模型的知识水平，这三个数据集分别是：自然问题 (Kwiatkowski 等人, 2019)、网络问题 (Berant 等人, 2013) 和琐事问答 (Joshi 等人, 2017)。闭卷指的是在没有补充参考资料或上下文材料的情况下提出的问题。为了评估模型的常识推理能力，我们使用 Winogrande 框架挑战 (Sakaguchi 等人, 2020) 对其进行测试。最后，我们在对抗性自然语言推理基准 (Nie 等人, 2019) 上测试模型的自然语言推理能力。

微调指标。本文通篇使用以下评估指标：我们报告了 GLUE 和 SuperGLUE 所有子任务的平均分数。Rouge-2 指标同时用于 CNNDM 和 XSum。在 SQuAD 和闭卷任务 (Web 问题、自然问题和琐事问题) 中，我们报告了答案与目标完全匹配的百分比 (有关此衡量标准的更多详细信息和不足，请参见 Roberts 等人 (2020))。最后，在 ARC Easy、ARC Challenge、ANLI 和 Winogrande 中，我们报告了生成响应的准确率。

微调结果。我们观察到在许多自然语言任务中，下游性能有显著提升。值得注意的改进来自 SuperGLUE，在该任务中，我们发现与 FLOP 匹配的 Switch 变体相比 T5-Base 和 T5-Large 基线分别提高了 4.4 和 2 个百分点，此外在 Winogrande、闭卷 Trivia QA 和 XSum 任务中也有大幅提升。在我们的微调研究中，唯一未观察到性能提升的任务是 AI2 推理挑战 (ARC) 数据集——在挑战数据集上，T5-Base 优于 Switch-Base；在简单数据集上，T5-Large 优于 Switch-Large。总体而言，我们观察到在推理任务和知识密集型任务中都有显著改进。这验证了我们的架构不仅预训练效果良好，还能通过微调将质量提升转化到下游任务中。

为了进行公平比较，我们的 T5 和 Switch 模型在修订后的 C4 数据集上进行了预训练，每批次包含 220 个 token，共训练了 550k 步。

Model	GLUE	SQuAD	SuperGLUE	Winogrande (XL)
T5-Base	84.3	85.5	75.1	66.6
Switch-Base	86.7	87.2	79.5	73.3
T5-Large	87.8	88.1	82.7	79.1
Switch-Large	88.5	88.6	84.7	83.0

Model	XSum	ANLI (R3)	ARC Easy	ARC Chal.
T5-Base	18.7	51.8	56.7	35.5
Switch-Base	20.3	54.0	61.3	32.8
T5-Large	20.9	56.6	68.8	35.5
Switch-Large	22.3	58.6	66.0	35.5

Model	CB Web QA	CB Natural QA	CB Trivia QA
T5-Base	26.6	25.8	24.5
Switch-Base	27.4	26.8	30.7
T5-Large	27.7	27.6	29.5
Switch-Large	31.3	29.5	36.9

表 5: 微调结果。T5 基准模型和 Switch 模型在多种自然语言测试（验证集；数值越高越好）中的微调结果。我们将计算量匹配的 Switch 模型与 T5-Base 和 T5-Large 基准模型进行了比较。在大多数所考虑的任务中，我们发现 Switch 变体有显著改进。我们观察到，无论是在模型规模上，还是在推理任务和知识密集型语言任务上，都有提升。

4.2 蒸馏

部署拥有数十亿乃至数万亿参数的大型神经网络并不方便。为缓解这一问题，我们研究了将大型稀疏模型蒸馏（Hinton 等人，2015）为小型密集模型的方法。未来的研究还可以进一步探索将大型模型蒸馏为更小的稀疏模型。

蒸馏技术。在表 6 中，我们研究了多种蒸馏技术。这些技术以 Sanh 等人（2019）的研究为基础，他们研究了 BERT 模型的蒸馏方法。我们发现，用非专家权重初始化密集模型会带来一定的改进。这是可能的，因为所有模型的 FLOP 都是匹配的，所以非专家层将具有相同的维度。由于专家层通常只在 Transformer 中的每个前馈网络（FFN）层或每隔一个 FFN 层添加，这使得许多权重可以用训练好的参数进行初始化。此外，我们观察到，使用 0.25 的教师概率和 0.75 的地面实况标签的混合比例时，蒸馏效果有所提升。通过结合这两种技术，我们用仅约 $1/20^{th}$ 的参数，获得了更大稀疏模型 30% 的质量增益（ $preserve \approx 30\%$ ）。质量增益指的是……的百分比。

Switch-Base（教师模型）和 T5-Base（学生模型）之间的质量差异。因此，100% 的质量提升意味着学生模型的性能与教师模型相当。

Technique	Parameters	Quality ($\uparrow$)
T5-Base	223M	-1.636
Switch-Base	3,800M	-1.444
Distillation	223M	(3%) -1.631
+ Init. non-expert weights from teacher	223M	(20%) -1.598
+ 0.75 mix of hard and soft loss	223M	(29%) -1.580
Initialization Baseline (no distillation)		
Init. non-expert weights from teacher	223M	-1.639

表 6: 用于语言建模的 Switch Transformer 蒸馏。用 Switch-Base 的非专家权重初始化 T5-Base, 并使用教师标签和真实标签混合产生的损失, 能获得最佳性能。我们可以将参数多 100 倍的大型稀疏模型 30% 的性能提升蒸馏回小型密集模型中。作为最终基线, 我们发现用专家权重初始化但不经过蒸馏而正常训练的 T5-Base 没有性能提升。

可实现的压缩率。使用表 6 中描述的最佳蒸馏技术, 我们将多种稀疏模型蒸馏为密集模型。我们蒸馏了 SwitchBase 版本, 通过增加专家数量进行全面测试, 这对应着参数在 11 亿到 147 亿之间变化。通过蒸馏, 对于 11 亿参数的模型, 我们可以保留 37% 的质量提升, 同时实现 82% 的压缩率。在极端情况下, 当我们模型压缩 99% 时, 仍然能够保持教师模型 28% 的质量提升。

对微调模型进行蒸馏。我们以将微调后的稀疏模型蒸馏为密集模型的研究作为结尾。表 8 展示了将在 SuperGLUE 任务上微调的 74 亿参数 Switch-Base 模型蒸馏到 2.23 亿参数 T5-Base 模型的结果。与我们的预训练结果类似, 我们发现当蒸馏到计算量匹配的密集变体时, 能够保留稀疏模型 30% 的性能提升。一个未在此处考虑的潜在未来方向, 可能是研究用于微调任务的特定专家, 并提取这些专家以实现更好的模型压缩。

4.3 多语言学习

在最后一组下游实验中, 我们在 101 种不同语言的混合数据上进行预训练时, 测量了模型质量和速度之间的权衡。我们以 mT5 (Xue 等人, 2020) 的最新研究为基础进行构建和基准测试, mT5 是 T5 的多语言扩展版本。我们在 mT5 中引入的涵盖 101 种语言的 Common Crawl 数据集的多语言变体 (mC4) 上进行预训练, 但由于某些语言存在文字变体, 该混合数据包含 107 个任务。

在图 7 中, 我们绘制了与 FLOP 匹配的 Switch 模型 (mSwitch-Base) 相对于 T5 基础变体 (mT5-Base) 在所有语言上的负对数困惑度质量提升。之后

	Dense	Sparse				
Parameters	223M	1.1B	2.0B	3.8B	7.4B	14.7B
Pre-trained Neg. Log Perp. ($\uparrow$)	-1.636	-1.505	-1.474	-1.444	-1.432	-1.427
Distilled Neg. Log Perp. ($\uparrow$)	—	-1.587	-1.585	-1.579	-1.582	-1.578
Percent of Teacher Performance	—	37%	32%	30 %	27 %	28 %
Compression Percent	—	82 %	90 %	95 %	97 %	99 %

表 7: 蒸馏压缩率。我们在将大型稀疏模型蒸馏为密集基线模型时对质量进行了测量。我们的基线模型 T5-Base 的负对数困惑度 (Neg. Log Perp.) 为 -1.636。在右侧列中, 我们将越来越大的稀疏模型蒸馏到同一个架构中。通过权重初始化以及硬损失和软损失的混合使用, 我们可以将稀疏教师模型压缩 95% 以上, 同时保留 30% 的质量增益。然而, 对于性能显著更优且规模更大的预训练教师模型, 我们认为需要更大的学生模型才能达到这些压缩率。

Model	Parameters	FLOPS	SuperGLUE ($\uparrow$)
T5-Base	223M	124B	74.6
Switch-Base	7410M	124B	81.3
Distilled T5-Base	223M	124B	(30%) 76.6

表 8: 提炼微调后的 SuperGLUE 模型。我们将在 SuperGLUE 任务上微调过的 Switch-Base 模型提炼为 T5-Base 模型。我们观察到, 在较小的数据集上, 我们的大型稀疏模型可以成为有效的提炼教师模型。我们发现, 在压缩 97% 的模型上, 我们再次实现了教师模型 30% 的性能。

在对两个版本进行 100 万步的预训练后, 我们发现, 在所有被研究的 101 种语言上, Switch Transformer 的最终负对数困惑度均高于基线模型。在图 8 中, 我们呈现了另一种视角, 展示了使用 Switch Transformer 相较于 mT5-Base 的每步加速比的直方图。我们发现, 相较于 mT5-Base, 其平均加速比为 5 倍, 且 91% 的语言实现了至少 4 倍的加速。这证明 Switch Transformer 是高效的多任务和多语言学习模型。

基于数据、模型和专家并行性设计模型

任意增加专家数量会受到收益递减的影响 (图 4)。在此我们介绍互补的扩展策略。扩展 Transformer 的常见方法是同步增加维度, 例如 d_{model} 或 d_{ff} 。这会同时增加参数

基于步骤的加速比计算方式为: 基准模型达到相同质量所需的步骤数除以我们的模型达到该质量所需的步骤数。

Switch Transformers

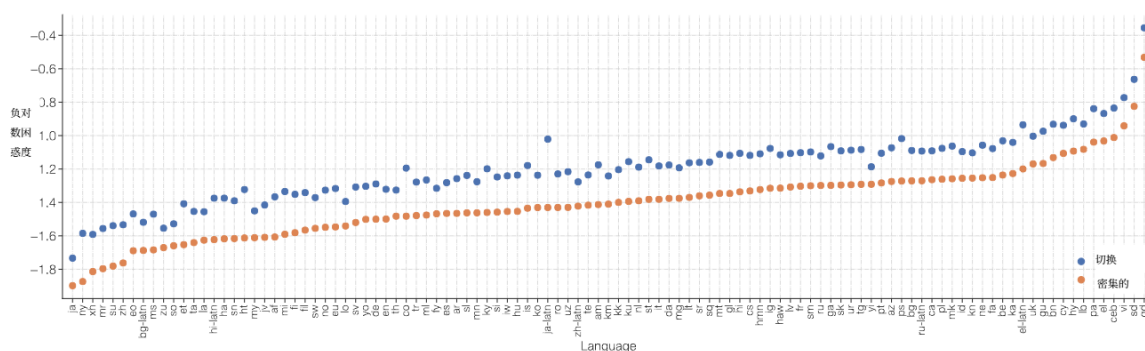


图 7: 101 种语言的多语言预训练。在 101 种语言的多任务训练中, Switch T5 Base 模型相对密集基线模型的改进。我们观察到 Switch Transformer 在多任务训练设置中表现相当出色, 并且在所有 101 种语言上都带来了改进。

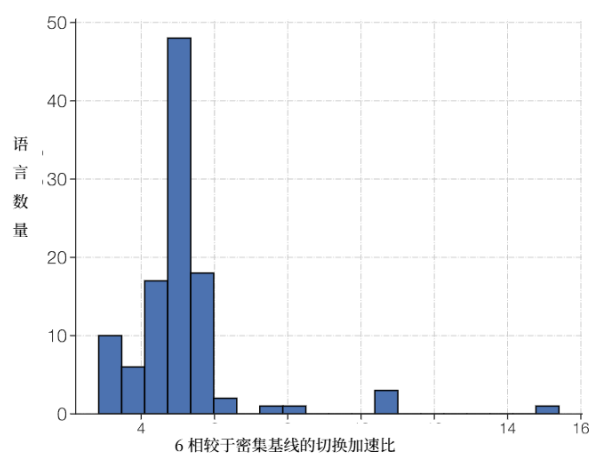


图 8: 101 种语言的多语言预训练。我们针对每种语言绘制了直方图, 展示了 Switch Transformer 相比与 FLOP 匹配的 T5 密集型基线模型在达到相同质量时的步数加速比。在所有 101 种语言上, 我们实现了相对于 mT5-Base 平均 5 倍的步数加速, 并且对于 91% 的语言, 在达到 mT5-Base 的最终困惑度时, 我们记录到 4 倍或更高的加速比。

以及所执行的计算, 最终受限于每个加速器的内存。一旦超过加速器的内存大小, 就可以采用单程序多数据 (SPMD) 模型并行。本节研究了数据并行、模型并行和专家并行相结合的权衡。

回顾前馈网络 (FFN) 层。我们以 FFN 层为例, 说明数据并行、模型并行和专家并行在 Mesh TensorFlow (Shazeer 等人, 2018) 中的工作原理, 并在此简要回顾。我们假设批次中有 B 个标记。每个标记的维度为

d_{model} 。FFN 的输入 ((x)) 和输出 ((y)) 的大小均为 $[B, d_{model}]$ ，中间值 ((h)) 的大小为 $[B, d_{ff}]$ ，其中 d_{ff} 通常比 d_{model} 大几倍。在 FFN 中，中间值经过 $h = xW_{in}$ 处理，然后该层的输出经过 $y = ReLU(h)W_{out}$ 处理。因此， W_{in} 和 W_{out} 独立应用于每个标记，其大小分别为 $[d_{model}, d_{ff}]$ 和 $[d_{ff}, d_{model}]$ 。

我们描述了分区的两个方面：权重和数据批次如何在核心上分配，如图 9 所示。我们将所有可用核心表示为 N ，Mesh TensorFlow 随后可能会将其重新映射到一个逻辑的多维处理器网格中。这里我们创建了一个二维逻辑网格，其中一个维度代表数据并行分片的方式数量 ((n))，另一个维度代表 ((m))。总核心数必须等于跨数据并行和模型并行的分片方式数量之和，例如 $N = n \times m$ 。为了在核心间分片该层，包含 B 个标记批次的张量会在 n 个数据并行核心上分片，因此每个核心包含 B/n 个标记。具有 d_{ff} 的张量和变量随后会在 m 个模型并行核心上分片。对于带有专家层的变体，我们考虑 E 个专家，每个专家最多可处理 c 个标记。

Term	Description
B	Number of tokens in the batch.
N	Number of total cores.
n	Number of ways for data-parallelism sharding.
m	Number of ways for model-parallelism sharding.
E	Number of experts in Switch layers.
C	Expert capacity, the batch size of each expert.

5.1 数据并行

在训练数据并行模型（这是分布式训练的标准方式）时，所有核心都分配给数据并行维度或 $n = N$ 、 $m = 1$ 。其优势在于，在整个前向和反向传播完成且需要在所有核心间聚合梯度之前，不需要进行任何通信。这对应于图 9 最左侧的列。

5.2 模型并行

我们现在考虑这样一种场景：所有核心都被专门分配给模型并行维度，因此 $n = 1$ ， $m = N$ 。现在，所有核心都必须保留完整的 B 令牌，并且每个核心将包含权重的一个独特切片。在每次前向和反向传递中，都会产生通信成本。每个核心发送一个 $[B, d_{model}]$ 张量来计算第二个矩阵乘法

$ReLU(h)W_{out}$ ，因为 d_{ff} 维度是分区的，必须进行求和。一般来说，只要跨核心分区的维度需要求和，就会在前向和反向传递中都添加一个全归约操作。这与纯数据并行形成对比，在纯数据并行中，全归约只在整个前向和反向传递结束时发生。

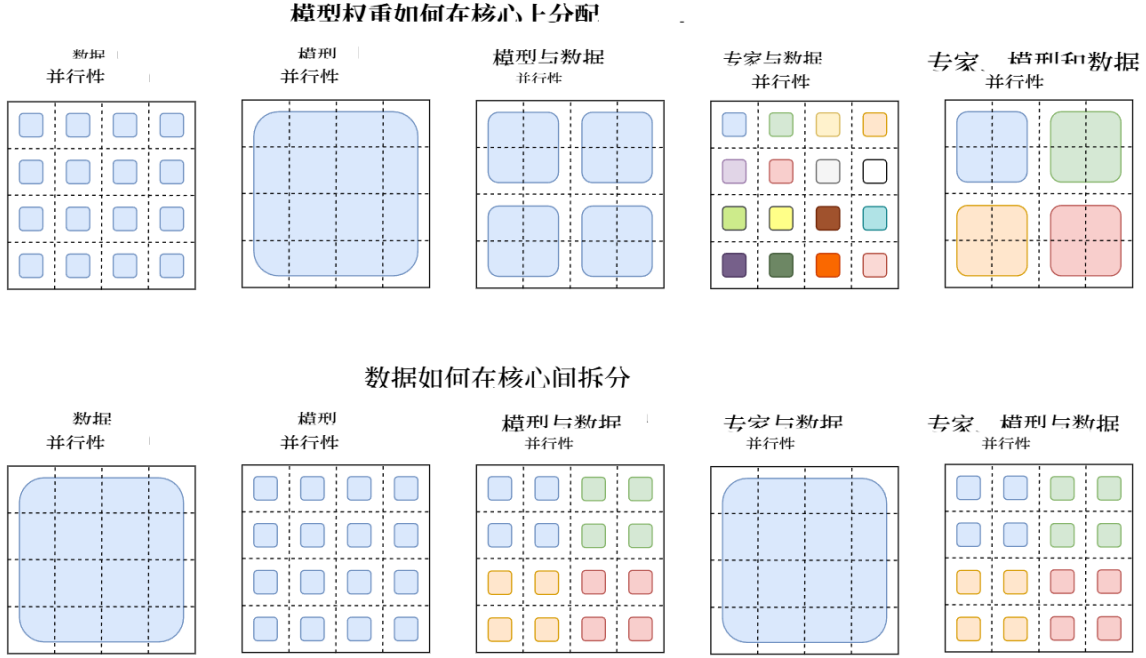


图 9：数据与权重分区策略。每个 4×4 虚线网格代表 16 个核心，阴影方块表示该核心上包含的数据（模型权重或令牌批次）。我们展示了每种策略下模型权重和数据张量的拆分方式。第一行：模型权重在各核心间的拆分示意图。此行中不同大小的形状代表前馈网络（FFN）层中更大的权重矩阵（例如更大的 d_{ff} 尺寸）。阴影方块的每种颜色标识一个独特的权重矩阵。每个核心的参数数量是固定的，但更大的权重矩阵会对每个令牌执行更多计算。第二行：数据批次在各核心间的拆分示意图。每个核心持有相同数量的令牌，这使得所有策略的内存使用保持固定。分区策略的不同之处在于，它们允许每个核心拥有相同的令牌或不同的令牌，这正是不同颜色所象征的含义。

5.3 模型与数据并行

对于大规模模型，混合使用模型并行和数据并行是很常见的，最大的 T5 模型（Raffel 等人，2019；Xue 等人，2020）和 GPT-3（Brown 等人，2020）都采用了这种方式。总共有 $N = n \times m$ 个核心，现在每个核心将负责 B/n 个标记以及权重和中间激活的 d_{ff}/m 部分。在正向和反向传播中，每个核心会在全归约操作中传递一个大小为 $\lceil B/n, d_{model} \rceil$ 的张量。

5.4 专家并行与数据并行

接下来，我们将描述专家并行和数据并行的分区策略。Switch Transformer 会将其所有核心分配到数据分区维度 n ，这也将对应模型中专家的数量。对于每个核心上的每个令牌，路由器会在本地计算与专家的分配关系。输出是一个大小为 $[n, B/n, E, C]$ 的二进制矩阵，该矩阵在第一维度上进行分区，并决定专家的分配。然后，这个二进制矩阵通过与 $[n, B/n, d_{model}]$ 、

$$\text{einsum}([n, B/n, d_{model}], [n, B/n, E, C], \text{dimension} = [B/n]) \quad (7)$$

的输入张量进行矩阵乘法来执行聚合操作，得到形状为 $[n, E, C, d_{model}]$ 的最终张量，该张量在第一维度上进行分片。由于每个核心都有自己的专家，我们执行一次大小为 $[E, C, d_{model}]$ 的全对全通信，以对 E 维度而非 n 维度进行分片。在前向传播中，还有大小为 $E \times C \times d_{model}$ 的 bfloat16 张量的额外通信成本，用于类似地接收位于不同核心上的每个专家的令牌。有关专家分区代码的详细分析，请参见附录 F。

5.5 专家、模型与数据并行

在我们最佳模型的设计中，我们力求平衡每个 token 的 FLOPs 与参数数量。当我们增加专家数量时，参数数量会随之增加，但每个 token 的 FLOPs 不会改变。为了提高 FLOPs，我们还必须增加 d_{ff} 维度（这也会增加参数，但增速较慢）。这就带来了一种权衡：随着我们增加 d_{ff} ，每个核心的内存将会耗尽，这就需要增加 m 。但由于我们的核心数量 N 是固定的，且 $N = n \times m$ ，我们必须减少 n ，这就迫使我们使用更小的批量大小（以保持每个核心的 token 数量不变）。

当结合模型并行和专家并行时，我们会面临全对全通信成本，这源于将令牌路由到正确的专家，同时还有来自模型并行的内部全归约通信。当结合这三种方法时，平衡每核的浮点运算次数、通信成本和内存变得相当复杂，此时最佳映射需要通过经验来确定。关于专家数量如何影响下游性能，也请参见我们在 5.6 节中的进一步分析。

5.6 迈向万亿参数模型

我们结合专家并行、模型并行和数据并行，设计了两个大型 Switch Transformer 模型，参数数量分别为 3950 亿和 1.6 万亿。我们研究了这些模型在作为语言模型的上游预训练及其下游微调中的表现。这两个不同模型的参数、每序列浮点运算次数 (FLOPs) 和超参数如下表 9 所示。文中描述了 Transformer 的标准超参数，包括 d_{model} 、 d_{ff} 、 d_{kv} 、头数和层数，以及一个不太常见的特征 $FFNNGEGLU$ ，其指的是前馈网络 (FFN) 层的一种变体，其中扩展矩阵被两组经非线性组合的权重所替代 (Shazeer, 2020)。

正如 5.4 节前面所描述的，Switch-C 模型仅采用专家并行设计，不使用模型并行。因此，控制宽度的超参数，

Model	Parameters	FLOPs/seq	d_{model}	FFN_{GELU}	d_{ff}	d_{kv}	Num. Heads
T5-Base	0.2B	124B	768	✓	2048	64	12
T5-Large	0.7B	425B	1024	✓	2816	64	16
T5-XXL	11B	6.3T	4096	✓	10240	64	64
Switch-Base	7B	124B	768	✓	2048	64	12
Switch-Large	26B	425B	1024	✓	2816	64	16
Switch-XXL	395B	6.3T	4096	✓	10240	64	64
Switch-C	1571B	890B	2080		6144	64	32

Model	Expert Freq.	Num. Layers	Num Experts	Neg. Log Perp. @250k	Neg. Log Perp. @ 500k
T5-Base	–	12	–	-1.599	-1.556
T5-Large	–	24	–	-1.402	-1.350
T5-XXL	–	24	–	-1.147	-1.095
Switch-Base	1/2	12	128	-1.370	-1.306
Switch-Large	1/2	24	128	-1.248	-1.177
Switch-XXL	1/2	24	64	-1.086	-1.008
Switch-C	1	15	2048	-1.096	-1.043

表 9: 切换模型设计与预训练性能。我们将 T5 模型的超参数和预训练性能与其切换 Transformer 变体进行了比较。最后两列分别记录了在 C4 数据集上经过 250k 和 500k 步训练后的预训练模型质量。我们发现, 在达到固定困惑度的情况下 (相同计算预算), SwitchC Transformer 变体的速度是 T5-XXL 模型的 4 倍, 且随着训练的进行, 这一差距还在扩大。

深度、头数等都比 T5-XXL 模型小得多。相比之下, Switch-XXL 与 T5-XXL 模型的 FLOP 相当, 这使得超参数可以有更大的维度, 但代价是模型并行性会带来额外的通信成本 (详见 5.5 节)。

与 T5-XXL 的样本效率对比。在表 9 的最后两列中, 我们分别记录了 250k 步和 500k 步后在 C4 语料库上的负对数困惑度。250k 步后, 我们发现两种 Switch Transformer 变体的负对数困惑度均比 T5-XXL 版本高出 0.06 以上 1.10。为了说明 0.06 这一差距的重要性, 我们注意到 T5-XXL 模型需要额外训练 250k 步才能提升 0.052。随着训练的继续, 这一差距不断扩大, 到 500k 步时, Switch-XXL 模型的表现比 T5-XXL 高出 0.087。

训练不稳定性。然而, 正如引言中所描述的, 大型稀疏模型可能不稳定, 而且随着规模的扩大, 我们会遇到一些偶发问题。我们发现, 拥有 1.6 万亿参数和 2048 个专家的更大的 Switch-C 模型完全没有表现出训练不稳定性。相反, 每个序列的 FLOPs 几乎大 10 倍的 Switch XXL 版本有时会不稳定。因此, 尽管这是我们在单步基础上表现更好的模型, 但我们并没有进行完整的 100 万步预训练, 这与 T5 的最终报告结果 (Raffel 等人, 2019 年) 一致。

这种报告的质量差异是一个下限, 实际差异可能更大。T5-XXL 是在更简单的 C4 数据集上预训练的, 该数据集包含重复的、因此容易被复制的示例片段。

推理微调性能。作为对模型质量的初步评估，我们使用了一个 Switch-XXL 模型，该模型在 5030 亿个 token 上进行了部分预训练，大约是 T5-XXL 模型所用文本量的一半。利用这个检查点，我们为了提高效率进行了多任务训练，所有任务都是联合学习的，而非单独微调。我们发现，验证集上的 SQuAD 准确率提升至 89.7，而最先进水平为 91.3。接下来，SuperGLUE 测试的平均得分记录为 87.5，相比之下，T5 版本的得分为 89.3，最先进水平为 90.0 (Wang 等人, 2019)。在 ANLI (Nie 等人, 2019) 上，Switch XXL 超过了之前的最先进水平，准确率达到 65.7，而之前的最佳成绩为 49.4 (Yang 等人, 2020)。我们注意到，尽管 SwitchXXL 在上游预训练任务上具有最先进的负对数困惑度 (Neg. Log Perp.)，但其优势尚未完全转化为下游任务的最先进性能。我们在附录 E 中对这一问题进行了更深入的研究。

基于知识的微调性能。最后，我们还在不使用显著跨度掩码 (Guu 等人, 2020) 进行额外预训练的情况下，通过三个闭卷知识任务对模型的知识进行了初步检验：自然问题、网络问题和琐事问答。在这三个任务中，我们都观察到模型性能优于之前最先进的 T5-XXL 模型 (不使用 SSM)。自然问题的精确匹配率从之前的最佳 32.8 提升至 34.4，网络问题从 37.2 提升至 41.0，琐事问答从 42.9 提升至 47.5。

总而言之，尽管训练数据量不到其他模型的一半，但我们已经发现该模型的质量具有可比性，有时甚至达到了最先进水平。目前，Switch Transformer 能将上游的显著收益更好地转化到基于知识的任务中，而非推理任务 (参见附录 E)。从大型专家模型中挖掘更强的微调性能是一个活跃的研究课题，而预训练的困惑度表明未来的改进是可能的。

相关工作

神经网络中规模的重要性已得到广泛认可，人们也提出了多种实现方法。近期的研究通过模型并行 (例如将权重和张量分配到多个核心上) 将模型规模扩展到数十亿参数 (Shazeer 等人, 2018; Rajbhandari 等人, 2019; Raffel 等人, 2019; Brown 等人, 2020; Shoeybi 等人, 2019)。此外，Harlap 等人 (2018)、Huang 等人 (2019) 提出了基于流水线的模型并行方法，即将不同的层分配到不同的设备上，并将微批次以流水线的方式传递到各个层。最后，产品密钥网络 (Lample 等人, 2019) 被提出，其通过根据输入到特定层的令牌表示查找可学习的嵌入，来提升神经网络的容量。

我们的研究针对一类条件计算方法中的特定模型展开，这类方法会基于输入动态做出计算决策。Cho 和 Bengio (2014) 提出根据模型隐藏状态中出现的特定位模式自适应地选择权重。Eigen 等人 (2013) 构建了带有密集矩阵乘法和 ReLU 激活函数的堆叠专家层，并在抖动 MNIST 和单调语音任务上展示了良好的结果。在计算机视觉领域，Puigcerver 等人 (2020) 在上游预训练阶段根据语义类别手动路由令牌，然后根据下游任务选择要使用的相关专家。

在现代深度学习架构中，专家混合模型（Mixture of Experts, MoE）的有效性已在 Shazeer 等人（2017 年）的研究中得到证实。该研究添加了一个 MoE 层，将其堆叠在长短期记忆网络（LSTM, Hochreiter 和 Schmidhuber, 1997 年）层之间，并且将标记分别路由到专家的组合中。这在语言建模和机器翻译基准测试中取得了最先进的结果。Mesh Tensorflow 库（Shazeer 等人, 2018 年）将 MoE 层重新引入 Transformer 架构，其中 MoE 层被用作前馈网络（FFN）层的替代方案，但没有附带自然语言处理（NLP）方面的结果。最近，随着机器学习基础设施的进步，扩展了 XLA 编译器的 GShard（Lepikhin 等人, 2020 年）使用 MoE Transformer 极大地改进了 100 种语言的机器翻译。最后，Fan 等人（2021 年）选择了一种不同的确定性 MoE 策略，将模型参数划分为不重叠的语言组。

Transformer 注意力模式中沿序列长度维度（ L ）的稀疏性已成为一种成功的技术，能够将注意力复杂度从 $O(L^2)$ 降低（Child 等人, 2019; Correia 等人, 2019; Sukhbaatar 等人, 2019; Kitaev 等人, 2020; Zaheer 等人, 2020; Beltagy 等人, 2020）。这使得学习比以往更长的序列成为可能。此版本的 Switch Transformer 没有采用注意力稀疏性，但这些技术具有互补性，并且作为未来的工作，它们可以结合起来，有可能改善在需要长上下文的任务中的学习效果。

讨论

我们提出并讨论了关于 Switch Transformer 以及一般稀疏专家模型的问题，这里的稀疏性指的是权重上的稀疏性，而非注意力模式上的。

Switch Transformer 难道不是因为参数数量多而更好吗？是的，而且这是设计使然！与所用的总 FLOPs 无关，参数是扩展神经语言模型的一个有用维度。大量研究已经充分表明，大型模型的性能更优（Kaplan 等人, 2020）。但在这种情况下，我们的模型在使用相同计算资源时，样本效率更高，速度也更快。

我没有超级计算机的使用权——这对我来说还有用吗？尽管这项研究聚焦于超大型模型，但我们发现，即便是仅有两个专家的模型也能提升性能，而且很容易满足常见 GPU 或 TPU 的内存限制（详见附录 D）。因此，我们认为我们的技术在小规模场景中也很有用。

在速度 - 精度帕累托曲线上，稀疏模型的表现优于密集模型吗？是的。在各种各样不同的模型规模下，稀疏模型在每一步以及实际运行时间上的表现都优于密集模型。我们的对照实验表明，在计算量和时间固定的情况下，稀疏模型的表现优于密集模型。

我无法部署万亿参数的模型——我们能缩小这些模型吗？我们无法完全保留模型质量，但通过将稀疏模型蒸馏为密集模型，可实现 10 到 100 倍的压缩率，同时保留专家模型 *achieving* $\approx 30\%$ 30% 的质量增益。

为什么要使用 Switch Transformer 而不是模型并行的密集型模型？从时间角度来看，Switch Transformer 比带有分片参数的密集型模型效率高得多（图 6）。此外，我们指出，这一选择并非互斥——我们

在 Switch Transformers 中，我们能够并且确实会使用模型并行技术，这增加了每个 token 的浮点运算次数，但也带来了传统模型并行技术的速度下降问题。

为什么稀疏模型尚未被广泛使用？尝试稀疏模型的动力受到了密集模型规模化巨大成功的阻碍（Hooker（2020）认为，密集模型的成功部分源于与深度学习硬件的协同适配）。此外，稀疏模型还存在诸多问题，包括（1）模型复杂度、（2）训练难度以及（3）通信成本。Switch Transformer 在缓解这些问题方面取得了进展。

未来工作

本文阐述了一种简化的架构、改进的训练流程，以及对稀疏模型如何扩展的研究。然而，仍有许多未来的开放方向，我们在此简要说明：

一个重大挑战是进一步提高最大型模型的训练稳定性。虽然我们的稳定性技术对 Switch-Base、Switch-Large 和 Switch-C 模型有效（未观察到不稳定性），但对于 Switch-XXL 来说还不够。我们已经采取了初步措施来稳定这些模型，我们认为这些措施可能对大型模型普遍有用，包括使用正则化器来提高稳定性以及采用适应性梯度裁剪形式，但这一问题仍未解决。

通常我们发现，预训练质量的提升会带来更好的下游任务结果（附录 E），不过我们有时也会遇到显著的异常情况。例如，尽管在对 C4 数据集进行建模时的困惑度相近，但 1.6T 参数的 Switch-C 在 SQuAD 中的精确匹配分数仅为 87.7，相比之下，规模更小的 Switch-XXL 模型则达到了 89.6。一个显著的差异是，Switch-XXL 模型的每 token 计算量（FLOPS）高于 Switch-C 模型，尽管其独特参数数量约为后者的 1/4（3950 亿对 1.6 万亿）。这表明，微调质量、每 token 计算量和参数数量之间存在一种尚未被充分理解的依赖关系。

对缩放关系进行全面研究，以指导融合数据并行、模型并行和专家并行的架构设计。理想情况下，给定硬件配置的规格（计算、内存、通信），人们可以更快速地设计出最优模型。反之，这也可能有助于未来硬件的设计。

我们的研究属于自适应计算算法范畴。我们的方法一直使用相同的、同质的专家，但未来的设计（借助更灵活的基础设施）可能会支持异质专家。这将通过在需要更多计算时（或许是针对更难的示例）路由到更大的专家，实现更灵活的适应。

研究 Transformer 的 FFN 层之外的专家层。我们发现初步证据表明，这同样可以提高模型质量。在附录 A 中，我们报告了将这些专家层添加到自注意力层内部所带来的质量提升，其中我们的

该层替换了生成 Q、K、V 的权重矩阵。然而，由于 bfloat16 格式存在训练不稳定性，我们将其留作未来的研究方向。

在新的模态以及不同模态间研究 Switch Transformer。到目前为止，我们仅考虑了语言，但我们认为模型稀疏性同样能在新的模态以及多模态网络中带来优势。

这份清单很容易扩展，但我们希望这能让大家对正在思考的各类挑战以及我们认为有前景的未来方向有所了解。

结论

Switch Transformers 是具有可扩展性且高效的自然语言学习模型。我们简化了混合专家模型，打造出一种易于理解、训练稳定且样本效率远高于同等规模密集型模型的架构。我们发现，这些模型在各类自然语言任务以及不同的训练模式（包括预训练、微调与多任务训练）中均表现出色。这些进展使得训练参数规模达数千亿至数万亿的模型成为可能，且相较于密集型 T5 基准模型，其速度有显著提升。我们希望我们的研究能推动稀疏模型成为一种高效的架构，并鼓励研究人员和从业者在自然语言任务及其他领域中考虑使用这些灵活的模型。

致谢

作者们要感谢玛格丽特·李，她在算法改进方面提供了数月的重要见解，并为实证研究提出了建议。感谢雨果·拉罗谢尔的明智指导和对草稿的清晰点评，感谢欧文·贝洛的详细意见和仔细修订，感谢科林·拉菲尔和亚当·罗伯茨就神经语言模型和 T5 代码库提供的及时建议，感谢约书亚·本吉奥在自适应计算研究方面的指导与鼓励，感谢雅沙·索尔-迪克斯坦为稳定新型大规模模型和论文修订提供的有趣新方向，以及谷歌大脑团队就论文展开的有益讨论。还要感谢布莱克·赫克特曼，他在分析和提升我们模型的训练性能方面提供了宝贵帮助。

A. 注意力切换

Shazeer 等人（2018 年）、Lepikhin 等人（2020 年）通过在 Transformer 的密集前馈网络（FFN）计算中加入 MoE 层，设计出了 MoE Transformer（Shazeer 等人，2017 年）。类似地，我们的研究也替换了 Transformer 中的 FFN 层，但在此我们简要探索一种不同的设计。我们将切换层加入到 Transformer 的自注意力层中。具体而言，我们用切换层替代了用于生成查询、键和值的可训练权重矩阵。如图 10 所示。

表 10 记录了几种变体在固定步数后的质量以及训练时间。尽管我们发现了改进，但也发现这些层在使用 bfloat16 精度时更不稳定，因此我们没有将它们纳入最终变体中。

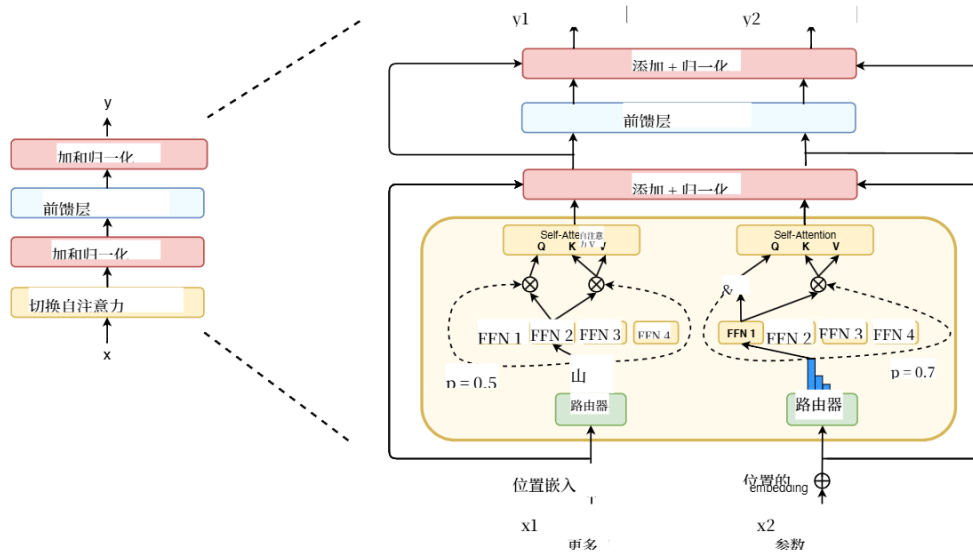


图 10: 注意力机制中的切换层。我们绘制了将切换层整合到自注意力 Transformer 块中的示意图。对于每个令牌（此处我们展示了两个令牌，x1 = “更多” 和 x2 = “参数”），一组权重生成查询，另一组独特的权重生成共享的键和值。我们测试了将每个专家作为线性操作以及前馈网络（FFN）的情况，这也是本研究中一直采用的方式。虽然我们发现这种方法能提升质量，但发现在使用低精度数字格式时会更不稳定，因此将其留待未来研究。

然而，当这些层能够稳定训练时，我们认为初步的积极结果预示着一个充满希望的未来方向。

Model	Precision	Quality @100k Steps (↑)	Quality @16H (↑)	Speed (ex/sec) (↑)
Experts FF	float32	-1.548	-1.614	1480
Expert Attention	float32	-1.524	-1.606	1330
Expert Attention	bfloat16	[diverges]	[diverges]	—
Experts FF + Attention	float32	-1.513	-1.607	1240
Expert FF + Attention	bfloat16	[diverges]	[diverges]	—

表 10: 切换注意力层的结果。所有模型都有 32 个专家，每个批次训练 524k 个 token。专家 FF 是指专家替换 Transformer 中的 FFN，这是我们在整篇论文中的标准设置。专家 FF + 注意力是指专家同时用于替换 FFN 和自注意力层。当使用 bfloat16 精度训练时，具有专家注意力的模型会出现发散。

B. 用“不落下任何 token”防止 token 丢失

由于 TPU 加速器的软件限制，我们的张量形状必须是静态大小的。因此，每个专家处理令牌表示的能力是有限且固定的。然而，这给我们的模型带来了一个问题，即模型在运行时动态路由令牌，可能导致令牌在专家之间的分布不均。如果发送给某个专家的令牌数量少于该专家的处理能力，那么计算过程可能只需进行填充——这虽然是对硬件的低效利用，但在数学上是正确的。但是，当发送给某个专家的令牌数量超过其处理能力（即专家溢出）时，就需要一种协议来处理这种情况。

Lepikhin 等人（2020 年）对混合专家模型进行了改进，通过残差连接将溢出令牌的表示传递到下一层，而不经该专家的处理，我们也采用了这种方法。

我们推测，不对令牌进行任何计算可能会非常浪费，尤其是当某个专家出现溢出时，这意味着另一个专家会有额外的容量。基于这一直觉，我们提出了“不让任何令牌掉队”（No-Token-Left-Behind）方法，该方法会迭代地重新路由那些最初被分配到溢出专家的令牌。图 11 展示了这种方法的图示说明，它能确保在训练和推理过程中几乎不会丢弃任何令牌。我们曾假设这可能会提高性能并进一步稳定训练，但并未发现实际的收益。我们推测，一旦网络学会了不同令牌与专家之间的关联，如果这种关联被改变（例如，将某个令牌发送给其第二优先级的专家），性能可能会下降。

C. 鼓励跨专家探索

在每个专家层，路由会决定将令牌发送给哪个专家。这是基于令牌表示信息，在可用专家中做出的离散决策。路由会根据传入的令牌表示确定最佳专家，但它不会收到关于选择其他专家可能表现如何的反事实信息。与强化学习一样，这里出现了经典的探索 - 利用困境（Sutton 和 Barto, 2018）。Rosenbaum 等人（2017）也注意到了这些问题，并采用了不同的解决方法，他们在多任务学习中取得了成功。这种特定场景与情境赌博机的场景最为接近（Robbins, 1952）。确定性地选择最优专家本质上是一种利用策略——我们考虑通过平衡探索来寻求更好的专家分配。

为了引入探索机制，我们考虑了几种方法：1) 确定性方法或最大值选取（argmax）2) 从 softmax 分布中采样 3) 对输入表示进行输入丢弃（input dropout）4) 对输入表示施加乘法抖动噪声。表 11 报告了这些方法对模型质量的最终影响。在整个研究过程中，我们使用输入抖动来注入噪声，因为经验表明这种方法的效果最佳。

D. 低计算量场景下的 Switch Transformer

Switch Transformer 在小规模场景以及拥有数千个核心和数万亿参数的情况下都是一种有效的架构。我们之前的许多实验都是

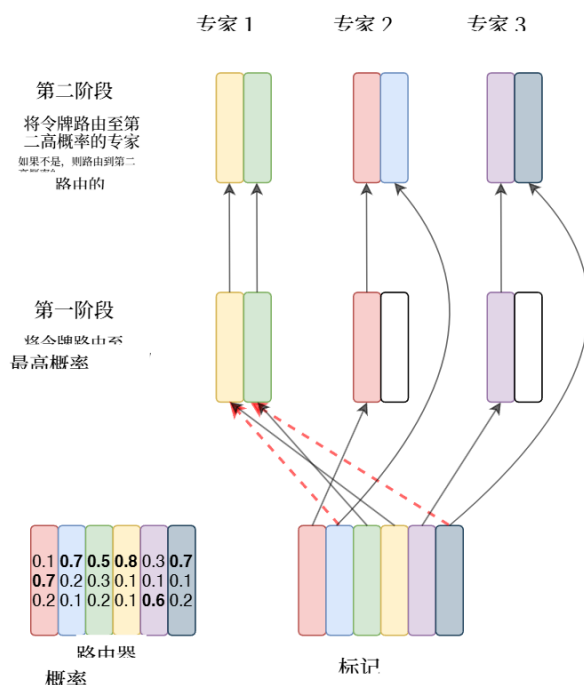


图 11：无令牌遗漏路由示意图。阶段 1 相当于交换机路由，其中令牌被路由到路由器给出的概率最高的专家。在阶段 2，我们查看所有溢出的令牌，并将它们路由到概率第二高的专家。如果其第二高概率的专家接收的令牌过多，令牌仍可能溢出，但这能让大多数令牌得到路由。这一过程可以迭代进行，以确保几乎不会有令牌被丢弃。

Model	Quality (Neg. Log Perp.) ($\uparrow$)
Argmax	-1.471
Sample softmax	-1.570
Input dropout	-1.480
Input jitter	-1.468

表 11：路由器探索策略。在不同的专家选择随机性策略下，通过负对数困惑度衡量的 Switch Transformer 质量（值越低越好）。这些变体之间在速度性能上没有实质性差异。

在 100 亿以上参数模型的规模下，但我们在图 12 中表明，仅用 2 个专家就比计算量相当的对应模型产生显著提升。即使没有现成的超级计算机，训练拥有 2、4 或 8 个专家的 Switch Transformer（我们通常建议每个核心配备一个专家），也能较 T5 密集型基线模型取得稳定改进。

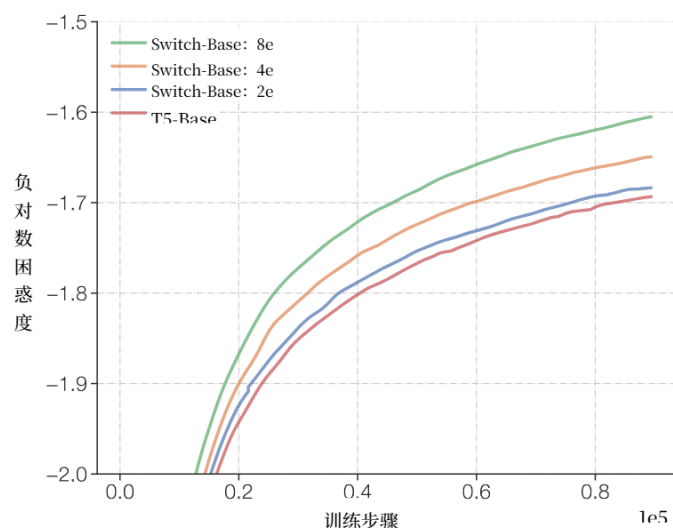


图 12: 专家数量较少的 Switch Transformer。即便专家数量极少, Switch Transformer 的性能仍优于基准模型。在此, 我们展示了其在极小规模下的缩放特性, 使用 2 个、4 个和 8 个专家时, 其性能均超过了 T5-Base 模型。

E. 上游与下游模型性能的关系

无法保证模型在预训练目标上的质量会转化为下游任务的结果。图 13 展示了密集模型和 Switch 模型在 C4 预训练任务上的上游模型质量与两个下游任务指标的相关性：SuperGLUE 的平均性能和 TriviaQA 得分。我们选择这两个任务是因为其中一个用于探究模型的推理能力，另一个用于探究模型的事实性知识。

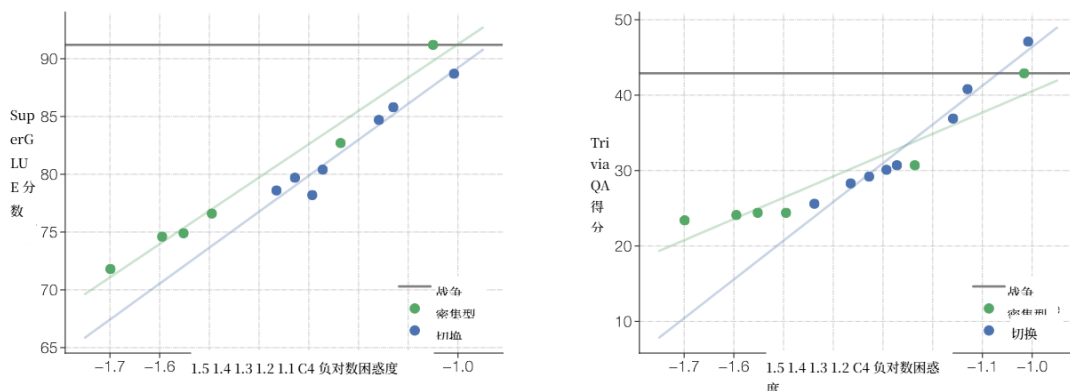


图 13：上游预训练质量与下游模型质量的关系。我们将上游性能与下游质量在 SuperGLUE 和 TriviaQA（未使用 SSM 记录的 SOTA）这两个分别侧重推理和知识的基准测试（验证集）上进行了关联分析。我们发现，与基线模型一样，Switch 模型的性能会随着上游预训练任务的改进而提升。在 SuperGLUE 上，我们发现负对数困惑度与 SuperGLUE 平均得分之间存在大致的线性关系。然而，在固定困惑度的情况下，密集型模型通常表现更优，尤其是在大规模场景中。相反，在知识密集型任务 TriviaQA 上，我们发现 Switch Transformer 可能遵循一种更优的缩放关系——在给定的上游困惑度下，它的表现优于对应的密集型模型。要确认这些观察结果，还需要更多的统计数据（收集成本较高，留待未来工作开展）。

我们发现了一种一致的相关性，这表明对于基线模型和 Switch 模型而言，改进的预训练会带来更好的下游任务结果。此外，在固定上游困惑度的情况下，我们发现 Switch 模型和密集模型在中小模型规模范围内的表现相近。然而，如 5.6 节所述，在最大模型规模（T5-11B/T5-XXL）下，我们最大的 Switch 模型并不总能将其上游困惑度很好地转化为在 SuperGLUE 任务上的下游微调效果。这值得未来进一步研究，以充分发挥稀疏模型的潜力。理解专家模型的微调动态非常复杂，它取决于正则化、负载均衡和微调超参数等因素。

F. Switch Transformers 的伪代码

Mesh Tensorflow 中 Switch Transformer 的伪代码 (Shazeer 等人, 2018)。以下代码未使用模型并行 (详见 5.4)。

```
import mesh_tensorflow as mtf

def load_balance_loss(router_probs, expert_mask):
    """Calculate load-balancing loss to ensure diverse expert routing."""
    # router_probs is the probability assigned for each expert per token.
    # router_probs shape: [num_cores, tokens_per_core, num_experts]
    # expert_index contains the expert with the highest router probability in one-hot format.
    # expert_mask shape: [num_cores, tokens_per_core, num_experts]

    # For each core, get the fraction of tokens routed to each expert.
    # density_l shape: [num_cores, num_experts]
    density_l = mtf.reduce_mean(expert_mask, reduced_dim=tokens_per_core)

    # For each core, get fraction of probability mass assigned to each expert
    # from the router across all tokens.
    # density_l_proxy shape: [num_cores, num_experts]
    density_l_proxy = mtf.reduce_mean(router_probs, reduced_dim=tokens_per_core)

    # density_l for a single core: vector of length num_experts that sums to 1.
    # density_l_proxy for a single core: vector of length num_experts that sums to 1.
    # Want both vectors to have uniform allocation (1/num_experts) across all num_expert elements.
    # The two vectors will be pushed towards uniform allocation when the dot product is minimized.
    loss = mtf.reduce_mean(density_l_proxy * density_l) * (num_experts ^ 2)
    return loss
```

图 14: Mesh Tensorflow 中 Switch Transformer 的负载平衡损失伪代码。

```

import mesh_tensorflow as mtf

def router(inputs, capacity_factor):
    """Produce the combine and dispatch tensors used for sending and
    receiving tokens from their highest probability expert. """
    # Core layout is split across num_cores for all tensors and operations.
    # inputs shape: [num_cores, tokens_per_core, d_model]

    router_weights = mtf.Variable(shape=[d_model, num_experts])

    # router_logits shape: [num_cores, tokens_per_core, num_experts]
    router_logits = mtf.einsum([inputs, router_weights], reduced_dim=d_model)

    if is_training:
        # Add noise for exploration across experts.
        router_logits += mtf.random_uniform(shape=router_logits.shape, minval=1-eps, maxval=1+eps)

    # Convert input to softmax operation from bfloat16 to float32 for stability.
    router_logits = mtf.to_float32(router_logits)

    # Probabilities for each token of what expert it should be sent to.
    router_probs = mtf.softmax(router_logits, axis=-1)

    # Get the top-1 expert for each token. expert_gate is the top-1 probability
    # from the router for each token. expert_index is what expert each token
    # is going to be routed to.
    # expert_gate shape: [num_cores, tokens_per_core]
    # expert_index shape: [num_cores, tokens_per_core]
    expert_gate, expert_index = mtf.top_1(router_probs, reduced_dim=num_experts)

    # expert_mask shape: [num_cores, tokens_per_core, num_experts]
    expert_mask = mtf.one_hot(expert_index, dimension=num_experts)

    # Compute load balancing loss.
    aux_loss = load_balance_loss(router_probs, expert_mask)

    # Experts have a fixed capacity, ensure we do not exceed it. Construct
    # the batch indices, to each expert, with position_in_expert
    # make sure that not more that expert_capacity examples can be routed to
    # each expert.
    position_in_expert = mtf.cumsum(expert_mask, dimension=tokens_per_core) * expert_mask

    # Keep only tokens that fit within expert_capacity.
    expert_mask *= mtf.less(position_in_expert, expert_capacity)
    expert_mask_flat = mtf.reduce_sum(expert_mask, reduced_dim=experts_dim)

    # Mask out the experts that have overflowed the expert capacity.
    expert_gate *= expert_mask_flat

    # combine_tensor used for combining expert outputs and scaling with router probability.
    # combine_tensor shape: [num_cores, tokens_per_core, num_experts, expert_capacity]
    combine_tensor = (
        expert_gate * expert_mask_flat *
        mtf.one_hot(expert_index, dimension=num_experts) *
        mtf.one_hot(position_in_expert, dimension=expert_capacity))

    # Cast back outputs to bfloat16 for the rest of the layer.
    combine_tensor = mtf.to_bfloat16(combine_tensor)

    # Create binary dispatch tensor that is 1 if the token gets routed to the corresponding expert.
    # dispatch_tensor shape: [num_cores, tokens_per_core, num_experts, expert_capacity]
    dispatch_tensor = mtf.cast(combine_tensor, tf.bool)

    return dispatch_tensor, combine_tensor, aux_loss

```

图 15: Mesh Tensorflow 中 Switch Transformers 的路由伪代码。

```

import mesh_tensorflow as mtf

def switch_layer(inputs, n, capacity_factor, num_experts):
    """Distributed switch transformer feed-forward layer."""
    # num_cores (n) = total cores for training the model (scalar).
    # d_model = model hidden size (scalar).
    # num_experts = total number of experts.
    # capacity_factor = extra buffer for each expert.
    # inputs shape: [batch, seq_len, d_model]
    batch, seq_len, d_model = inputs.get_shape()

    # Each core will route tokens_per_core tokens to the correct experts.
    tokens_per_core = batch * seq_len / num_cores

    # Each expert will have shape [num_cores, expert_capacity, d_model].
    # Each core is responsible for sending expert_capacity tokens
    # to each expert.
    expert_capacity = tokens_per_core * capacity_factor / num_experts

    # Reshape to setup per core expert dispatching.
    # shape: [batch, seq_len, d_model] -> [num_cores, tokens_per_core, d_model]
    # Core layout: [n, 1, 1] -> [n, 1, 1]
    inputs = mtf.reshape(inputs, [num_cores, tokens_per_core, d_model])

    # Core Layout: [n, 1, 1] -> [n, 1, 1, 1], [n, 1, 1, 1]
    # dispatch_tensor (boolean) shape: [num_cores, tokens_per_core, num_experts, expert_capacity]
    # dispatch_tensor is used for routing tokens to the correct expert.
    # combine_tensor (float) shape: [num_cores, tokens_per_core, num_experts, expert_capacity]
    # combine_tensor used for combining expert outputs and scaling with router
    # probability.
    dispatch_tensor, combine_tensor, aux_loss = router(inputs, expert_capacity)

    # Matmul with large boolean tensor to assign tokens to the correct expert.
    # Core Layout: [n, 1, 1], -> [1, n, 1, 1]
    # expert_inputs shape: [num_experts, num_cores, expert_capacity, d_model]
    expert_inputs = mtf.einsum([inputs, dispatch_tensor], reduce_dims=[tokens_per_core])

    # All-to-All communication. Cores split across num_cores and now we want to split
    # across num_experts. This sends tokens, routed locally, to the correct expert now
    # split across different cores.
    # Core layout: [1, n, 1, 1] -> [n, 1, 1, 1]
    expert_inputs = mtf.reshape(expert_inputs, [num_experts, num_cores, expert_capacity, d_model])

    # Standard feed forward computation, where each expert will have its own
    # unique set of parameters.
    # Total unique parameters created: num_experts * (d_model * d_ff * 2).
    # expert_outputs shape: [num_experts, num_cores, expert_capacity, d_model]
    expert_outputs = feed_forward(expert_inputs)

    # All-to-All communication. Cores are currently split across the experts
    # dimension, which needs to be switched back to being split across num_cores.
    # Core Layout: [n, 1, 1, 1] -> [1, n, 1, 1]
    expert_outputs = mtf.reshape(expert_outputs, [num_experts, num_cores, expert_capacity, d_model])

    # Convert back to input shape and multiply outputs of experts by the routing probability.
    # expert_outputs shape: [num_experts, num_cores, tokens_per_core, d_model]
    # expert_outputs.combined shape: [num_cores, tokens_per_core, d_model]
    # Core Layout: [1, n, 1, 1] -> [n, 1, 1]
    expert_outputs.combined = mtf.einsum([expert_outputs, combine_tensor], reduce_dims=[tokens_per_core])

    # Remove tokens_per_core shapes used for local routing dispatching to match input shape.
    # Core Layout: [n, 1, 1] -> [n, 1, 1]
    outputs = mtf.reshape(expert_outputs.combined, [batch, seq_len, d_model])
    return outputs, aux_loss

```

图 16: Mesh TensorFlow 中 Switch Transformer 层的伪代码。

参考文献

- Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard 等人。TensorFlow: 一个大规模机器学习系统。发表于第 12 届 {USENIX} 操作系统设计与实现研讨会 ({OSDI} 16), 第 265–283 页, 2016 年。
- Iz Beltagy, Matthew E Peters 和 Arman Cohan。《Longformer: 长文档转换器》。arXiv 预印本 arXiv:2004.05150. 2020 年。
- 乔纳森·贝兰特、安德鲁·周、罗伊·弗罗斯蒂希和珀西·梁。基于问答对的 Freebase 语义解析。见《2013 年自然语言处理经验方法会议论文集》, 第 1533-1544 页, 2013 年。
- 汤姆·B·布朗、本杰明·曼、尼克·赖德、梅兰妮·萨比亚、贾里德·卡普兰、普拉富拉·达里瓦尔、阿尔温德·尼拉坎坦、普拉纳夫·希亚姆、吉里什·萨斯特里、阿曼达·阿斯科尔等人。语言模型是少样本学习者。arXiv 预印本 arXiv:2005.14165. 2020 年。
- Rewon Child, Scott Gray, Alec Radford 和 Ilya Sutskever。《使用稀疏 Transformer 生成长序列》。arXiv 预印本 arXiv:1904.10509. 2019 年。
- Kyunghyun Cho 和 Yoshua Bengio。《深度学习中条件计算的容量与计算比的指数级提升》。arXiv 预印本 arXiv:1406.7362. 2014 年。
- 彼得·克拉克、艾萨克·考伊、奥伦·埃齐奥尼、图沙尔·霍特、阿希什·萨巴瓦尔、卡丽莎·舍尼克和奥伊文德·塔夫约德。认为自己解决了问答问题? 来试试 ARC, 即 AI2 推理挑战。arXiv 预印本 arXiv:1803.05457. 2018 年。
- Gonçalo M Correia, Vlad Niculae 和 André FT Martins。自适应稀疏 Transformer。arXiv 预印本 arXiv:1909.00015. 2019。
- 雅各布·德夫林、明伟·张、肯顿·李和克里斯蒂娜·托坦诺娃。Bert: 用于语言理解的深度双向 Transformer 预训练。arXiv 预印本 arXiv:1810.04805, 2018 年。
- 大卫·艾根、马克·奥雷里奥·兰扎托和伊利亚·萨茨克弗。深度混合专家模型中的因子分解表示学习。arXiv 预印本 arXiv:1312.4314. 2013 年。
- 安吉拉·范、施鲁蒂·博萨莱、霍尔格·施文克、马志毅、艾哈迈德·埃尔-基什基、悉达多·戈亚尔、曼迪普·贝恩斯、奥努尔·切莱比、纪尧姆·文泽克、维什拉夫·乔杜里等人。超越以英语为中心的多语言机器翻译。《机器学习研究期刊》, 2021 年, 第 22 卷 (107 期): 1-48。
- William Fedus, Ian Goodfellow 和 Andrew M Dai。Maskgan: 通过填充来实现更好的文本生成。arXiv 预印本 arXiv:1801.07736. 2018 年。
- 特雷弗·盖尔、马泰·扎哈里亚、克利夫·扬和埃里希·埃尔森。深度学习的稀疏 GPU 内核。arXiv 预印本 arXiv:2006.10901, 2020 年。
- 斯科特·格雷、亚历克·拉德福德和迪德里克·P·金马。用于块稀疏权重的 GPU 内核。<https://openai.com/blog/block-sparse-gpu-kernels/>. 2017 年。

Kelvin Guu、Kenton Lee、Zora Tung、Panupong Pasupat 和 Ming-Wei Chang。Realm：检索增强的语言模型预训练。arXiv 预印本 arXiv:2002.08909, 2020。

Aaron Harlap、Deepak Narayanan、Amar Phanishayee、Vivek Seshadri、Nikhil Devanur、Greg Ganger 和 Phil Gibbons。Pipedream：快速高效的管道并行 DNN 训练。arXiv 预印本 arXiv:1806.03377, 2018 年。

卡尔·莫里茨·赫尔曼、托马斯·科奇斯基、爱德华·格雷芬斯特特、拉塞·埃斯佩霍尔特、威尔·凯、穆斯塔法·苏莱曼和菲尔·布伦索姆。《教机器阅读与理解》。收录于 C. 科尔特斯、N. 劳伦斯、D. 李、M. 杉山和 R. 加内特编辑的《神经信息处理系统进展》第 28 卷，第 1693-1701 页。Curran Associates, Inc., 2015 年。网址：
<https://proceedings.neurips.cc/paper/2015/file/afdec7005cc9f14302cd0474fd0f3c96-Paper.pdf>。

杰弗里·辛顿、奥里奥尔·文尼亚斯和杰夫·迪恩。《提炼神经网络中的知识》。arXiv 预印本 arXiv:1503.02531, 2015 年。

塞普·霍克里特和约尔根·施密德胡伯。长短期记忆。《神经计算》，9 (8): 1735–1780, 1997。

萨拉·胡克。硬件 lottery。arXiv 预印本 arXiv:2009.06489, 2020。

黄艳平、程有龙、安库尔·巴普纳、奥尔汗·菲拉特、陈德豪、陈米娅、李亨钟、吴季泉、黎国光、吴永辉等人。《GPipe：使用流水线并行高效训练巨型神经网络》。收录于《神经信息处理系统进展》，第 103-112 页，2019 年。

罗伯特·A·雅各布斯、迈克尔·I·乔丹、史蒂文·J·诺兰和杰弗里·E·辛顿。局部专家的自适应混合。《神经计算》，3 (1): 79–87, 1991。

迈克尔·I·乔丹和罗伯特·A·雅各布斯。专家分层混合模型与 EM 算法。《神经计算》，6 (2):181–214, 1994。

曼达尔·乔希、恩索尔·崔、丹尼尔·S·韦尔德和卢克·泽特勒莫耶。Triviaqa：一个用于阅读理解的大规模远程监督挑战数据集。arXiv 预印本 arXiv:1705.03551, 2017 年。

贾里德·卡普兰、山姆·麦坎德利斯、汤姆·亨尼根、汤姆·B·布朗、本杰明·切斯、雷旺·蔡尔德、斯科特·格雷、亚历克·拉德福德、杰弗里·吴和达里奥·阿莫代伊。神经语言模型的缩放定律。arXiv 预印本 arXiv:2001.08361, 2020 年。

尼基塔·基塔耶夫、卢卡斯·凯泽和安塞尔姆·列夫斯卡娅。《Reformer：高效 Transformer》。arXiv 预印本 arXiv:2001.04451, 2020 年。

汤姆·克维亚特科夫斯基、詹妮玛丽亚·帕洛马基、奥利维亚·雷德菲尔德、迈克尔·柯林斯、安库尔·帕里赫、克里斯·阿尔贝蒂、丹妮尔·爱泼斯坦、伊利亚·波洛苏欣、雅各布·德夫林、肯顿·李等人。自然问题：问答研究的基准。《计算语言学协会会刊》，7:453–466, 2019。

纪尧姆·兰普尔、亚历山大·萨布莱罗勒、马克·奥雷利奥·兰扎托、吕多维克·德诺耶尔和埃尔韦·热古。带有乘积键的大型记忆层。《神经信息处理系统进展》，第 8548–8559 页，2019 年。

Katherine Lee、Daphne Ippolito、Andrew Nystrom、Chiyuan Zhang、Douglas Eck、Chris Callison-Burch 和 Nicholas Carlini。去除训练数据中的重复项能让语言模型变得更好。arXiv 预印本 arXiv:2107.06499, 2021 年。

德米特里·列皮欣、李亨宗、徐元忠、陈德浩、奥尔汉·菲拉特、黄燕平、马克西姆·克里昆、诺姆·沙泽尔和陈志峰。《GShard: 通过条件计算和自动分片扩展巨型模型》。arXiv 预印本 arXiv:2006.16668, 2020 年。

保利乌斯·米西克维丘斯、沙兰·纳兰、乔纳·阿尔本、格雷戈里·迪亚莫斯、埃里希·埃尔森、大卫·加西亚、鲍里斯·金斯堡、迈克尔·休斯顿、奥莱克西·库恰耶夫、加内什·文卡特什等人。混合精度训练。arXiv 预印本 arXiv:1710.03740, 2017 年。

沙希·纳拉扬、谢伊·B·科恩和米雷拉·拉帕塔。《别给我细节，只要摘要！用于极端摘要的主题感知卷积神经网络》。arXiv 预印本 arXiv:1808.08745, 2018 年。

聂一心、阿迪娜·威廉姆斯、艾米丽·迪南、莫希特·班萨尔、贾森·韦斯顿和杜威·基拉。对抗性自然语言推理：自然语言理解的新基准。arXiv 预印本 arXiv:1910.14599, 2019 年。

琼安·普伊格塞尔韦、卡洛斯·里克尔梅、巴兹尔·穆斯塔法、塞德里克·伦格利、安德烈·苏萨诺·平托、西尔万·盖利、丹尼尔·凯泽斯和尼尔·霍尔斯基。《使用专家模型进行可扩展迁移学习》。arXiv 预印本 arXiv:2009.13239, 2020 年。

亚历克·拉德福德、卡尔西克·纳拉辛汉、蒂姆·萨利曼斯和伊利亚·萨茨克弗。通过生成式预训练提升语言理解。2018 年。

科林·拉菲尔、诺阿姆·沙泽尔、亚当·罗伯茨、凯瑟琳·李、沙兰·纳朗、迈克尔·马特纳、周彦琦、李伟和彼得·J·刘。利用统一的文本到文本转换器探索迁移学习的极限。arXiv 预印本 arXiv:1910.10683, 2019 年。

Samyam Rajbhandari、Jeff Rasley、Olatunji Ruwase 和 Yuxiong He。《Zero: 面向训练万亿参数模型的内存优化》。arXiv 预印本 arXiv:1910.02054, 2019。

普拉纳夫·拉杰普尔卡、张建、康斯坦丁·洛皮列夫和珀西·梁。Squad: 100,000 多个用于文本机器理解的问题。arXiv 预印本 arXiv:1606.05250, 2016 年。

Prajit Ramachandran 和 Quoc V Le。示例路由模型的多样性与深度。发表于《国际学习表征会议》。2018 年。

赫伯特·罗宾斯。实验序贯设计的若干方面。《美国数学会通报》，58 (5): 527–535, 1952。

亚当·罗伯茨、科林·拉菲尔和诺姆·沙泽尔。你能在语言模型的参数中融入多少知识？arXiv 预印本 arXiv:2002.08910, 2020 年。

克莱门斯·罗森鲍姆、蒂姆·克林格和马修·里默。路由网络：多任务学习中非线性函数的自适应选择。arXiv 预印本 arXiv:1711.01239, 2017 年。

Keisuke Sakaguchi、Ronan Le Bras、Chandra Bhagavatula 和 Yejin Choi。Winogrande：大规模的对抗性 Winograd 模式挑战。见《AAAI 人工智能大会论文集》，第 34 卷，第 8732-8740 页。2020 年。

维克多·桑（Victor Sanh）、利桑德·德布（Lysandre Debut）、朱利安·肖蒙德（Julien Chaumond）和托马斯·沃尔夫（Thomas Wolf）。《DistilBERT：BERT 的蒸馏版本——更小、更快、更经济、更轻量》，2019 年。

诺姆·沙泽尔。Glu 变体改讲 Transformer。2020 年。

诺姆·沙泽尔、阿扎利亚·米尔霍塞尼、克日什托夫·马齐亚尔兹、安迪·戴维斯、曲克·勒·杰弗里·辛顿和杰夫·迪恩。《超大神经网络：稀疏门控混合专家层》。arXiv 预印本 arXiv:1701.06538, 2017 年。

Noam Shazeer、Youlong Cheng、Niki Parmar、Dustin Tran、Ashish Vaswani、Penporn Koanantakool、Peter Hawkins、HyoukJoong Lee、Mingsheng Hong、Cliff Young 等人。《Mesh-tensorflow：超级计算机的深度神经网络》。发表于《神经信息处理系统进展》，第 10414-10423 页，2018 年。

穆罕默德·舒埃比、莫斯托法·帕特瓦里、劳尔·普里、帕特里克·勒格雷斯科、贾里德·卡斯珀和布莱恩·卡坦扎罗。Megatron-LM：使用 GPU 模型并行训练数十亿参数的语言模型。arXiv 预印本 arXiv:1909.08053, 2019 年。

尼蒂什·斯里瓦斯塔瓦、杰弗里·E·辛顿、亚历克斯·克里热夫斯基、伊利亚·萨茨克韦弗和鲁斯兰·萨拉赫丁诺夫。Dropout：一种防止神经网络过拟合的简单方法。《机器学习研究期刊》，15 (1):1929-1958, 2014。网址 <http://www.cs.toronto.edu/~rsalakhu/papers/srivastava14a.pdf>。

艾玛·斯特鲁贝尔、阿南娅·加内什和安德鲁·麦卡勒姆。自然语言处理中深度学习的能源与政策考量。arXiv 预印本 arXiv:1906.02243, 2019。

Sainbayar Sukhbaatar、Edouard Grave、Piotr Bojanowski 和 Armand Joulin。Transformer 中的自适应注意力跨度。arXiv 预印本 arXiv:1905.07799, 2019 年。

里奇·萨顿。《惨痛的教训》。<http://www.incompleteideas.net/IncIdeas/BitterLesson.html>, 2019。

理查德·S·萨顿和安德鲁·G·巴托。《强化学习：导论》。斯坦福大学，2018 年。

威尔逊·L·泰勒。《“完形填空法”：一种衡量可读性的新工具》。《新闻季刊》，第 30 卷（第 4 期）：415-433 页。1953 年。

阿希什·瓦斯瓦尼、诺姆·沙泽尔、尼基·帕尔马、雅各布·乌兹科雷特、利昂·琼斯、艾丹·N
《注意力就是一切》。见《神经信息处理系统进展》，第 5998-6008 页，2017 年。

亚历克斯·王、阿曼普雷特·辛格、朱利安·迈克尔、费利克斯·希尔、奥默·利维以及塞缪尔·
R·鲍曼。GLUE：一个用于自然语言理解的多任务基准和分析平台。arXiv 预印本
arXiv:1804.07461。2018 年。

亚历克斯·王、亚达·普鲁克萨查特昆、尼基塔·南吉亚、阿曼普雷特·辛格、朱利安·迈克尔、
费利克斯·希尔、奥默·利维以及塞缪尔·鲍曼。《SuperGLUE：一个更具挑战性的通用语言理解
系统基准》。收录于《神经信息处理系统进展》，第 3266-3280 页，2019 年。

王世博和潘卡伊·坎瓦尔。Bfloat16：云 TPU 高性能的秘诀。《谷歌云博客》，2019 年。

Linting Xue、Noah Constant、Adam Roberts、Mihir Kale、Rami Al-Rfou、Aditya
Siddhant、Aditya Barua 和 Colin Raffel。mt5：一个大规模多语言预训练文本到文本转换器。
arXiv 预印本 arXiv:2010.11934，2020。

杨智林、戴子航、杨一鸣、海梅·卡博内尔、鲁斯兰·萨拉赫丁诺夫和黎国柱。《XLNet：用于语言
理解的广义自回归预训练》，2020 年。

曼 zil·扎希尔、古鲁·古鲁加内什、阿维纳瓦·杜贝、约书亚·安斯利、克里斯·阿尔贝蒂、圣
地亚哥·翁塔农、菲利普·范·阿尼鲁德·拉武拉、王启帆、李阳等人。大鸟：用于更长序列的
Transformer。arXiv 预印本 arXiv:2007.14062。2020 年。